

FUSE Introductory Tutorial

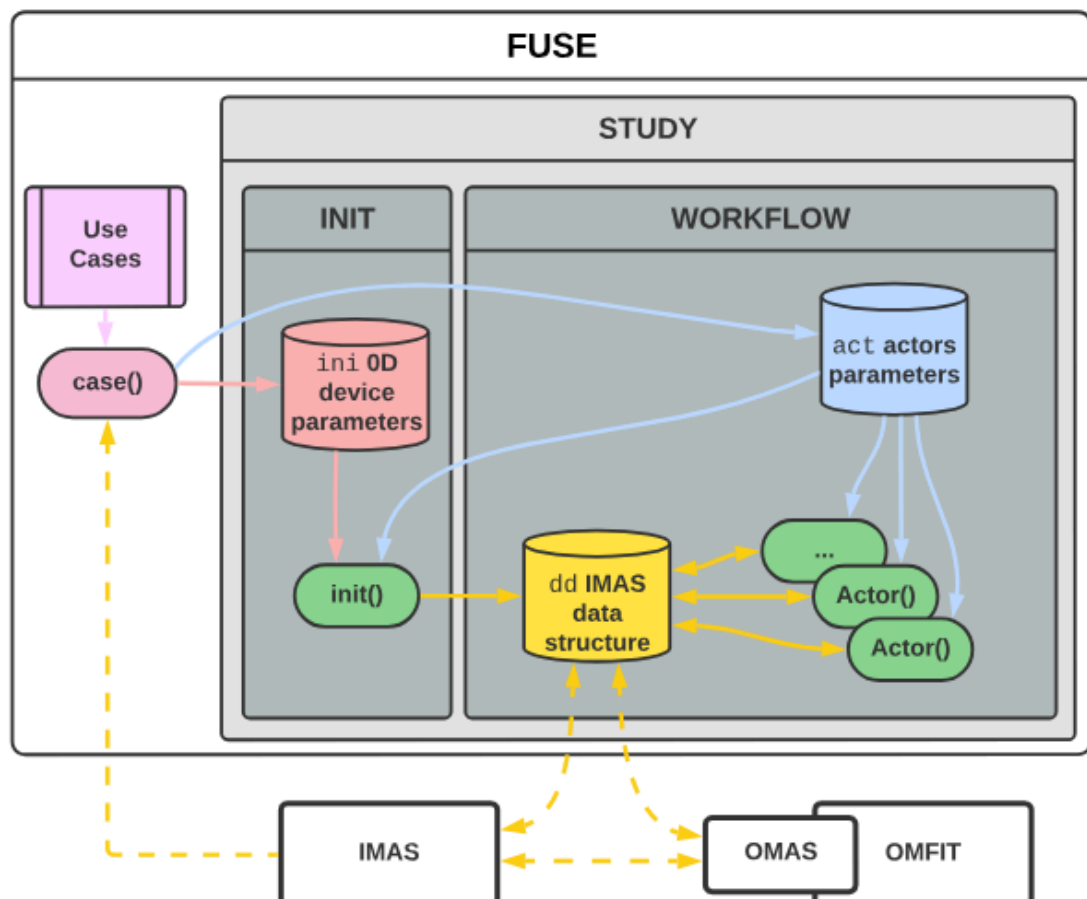
Download this tutorial from the [FuseExamples repository](#)

Basic concepts

To make sense of this tutorial, you'll need to know the following organization concepts of FUSE:

1. 📁 **Data storage:** All data is stored in the `dd` structure, which follows the ITER IMAS ontology.
2. 🧠 **Actors:** The core components of FUSE simulations are physics and engineering actors.
3. 🛠️ **Control:** Actor functionality is governed by `act` parameters.
4. 🚀 **Initialization:** The data structure can be initialized from OD `ini` parameters.
5. 🔧 **Use cases:** FUSE includes templates for various machines (e.g., FPP, ITER, ARC).
6. 🔄 **Workflows:** Self-contained studies and optimizations are conducted via workflows, typically involving multiple FUSE simulations.
7. 🌐 **Interoperability:** FUSE interfaces with existing modeling tools like OMFIT/OMAS and the IMAS ecosystem.

A diagram illustrating these concepts is provided below:



Let's get started!

NOTE: Julia is a Just In Time (JIT) programming language. The first time something is executed it will take longer because of the compilation process. Subsequent calls the the same code will be blazingly fast.

Import the necessary packages

```
In [2]: using Plots # for plotting
        using FUSE # this will also import IMAS in the current namespace
```

```
[ Info: Precompiling FUSE [e64856f0-3bb8-4376-b4b7-c03396503992]
[ Info: Precompiling FUSE [e64856f0-3bb8-4376-b4b7-c03396503992]
```

```
SYSTEM: caught exception of type :MethodError while trying to print a failed Task
notice; giving up
```

Starting from a use-case

FUSE comes with some predefined [use-cases](#), some of which are used for regression testing.

Note that some use cases are for non-nuclear experiments and certain actors like Blankets or BalanceOfPlant will not perform any actions.

Get initial parameters (`ini`) and actions (`act`) for a given use-case, let's use KDEMO for example

```
In [3]: ini, act = FUSE.case_parameters(:KDEMO);
```

The `ini` data structure contains OD parameters that will be used to bootstrap the `dd` with plausible data.

The `ini` [parameters](#) can be modified.

```
In [4]: ini.equilibrium.B0 = 7.8
        ini.equilibrium.R0 = 6.5;
```

The `act` data structure contains parameters that define how the actors (ie the models) will behave.

The `act` [parameters](#) can also be modified.

```
In [5]: act.ActorCoreTransport.model = :FluxMatcher;
```

`ini` and `act` can now be used to initialize the data dictionary (`dd`) using the OD parameters.

NOTE: `init()` does not return a self-consistent solution, just a plausible starting point to initialize our simulations!

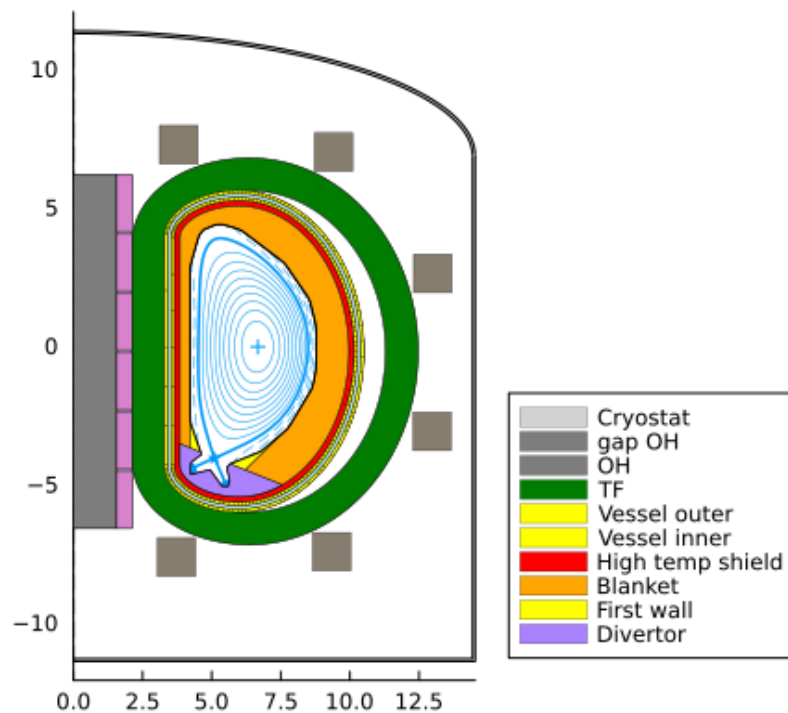
```
In [6]: dd = IMAS.dd() # an empty dd
        FUSE.init(dd, ini, act);
```

```
actors: Equilibrium
actors: TEQUILA
actors: CXbuild
actors: Sources
actors: SimpleEC
actors: SimpleIC
actors: NeutralFueling
actors: Current
actors: QED
actors: PassiveStructures
```

Let's see what we got

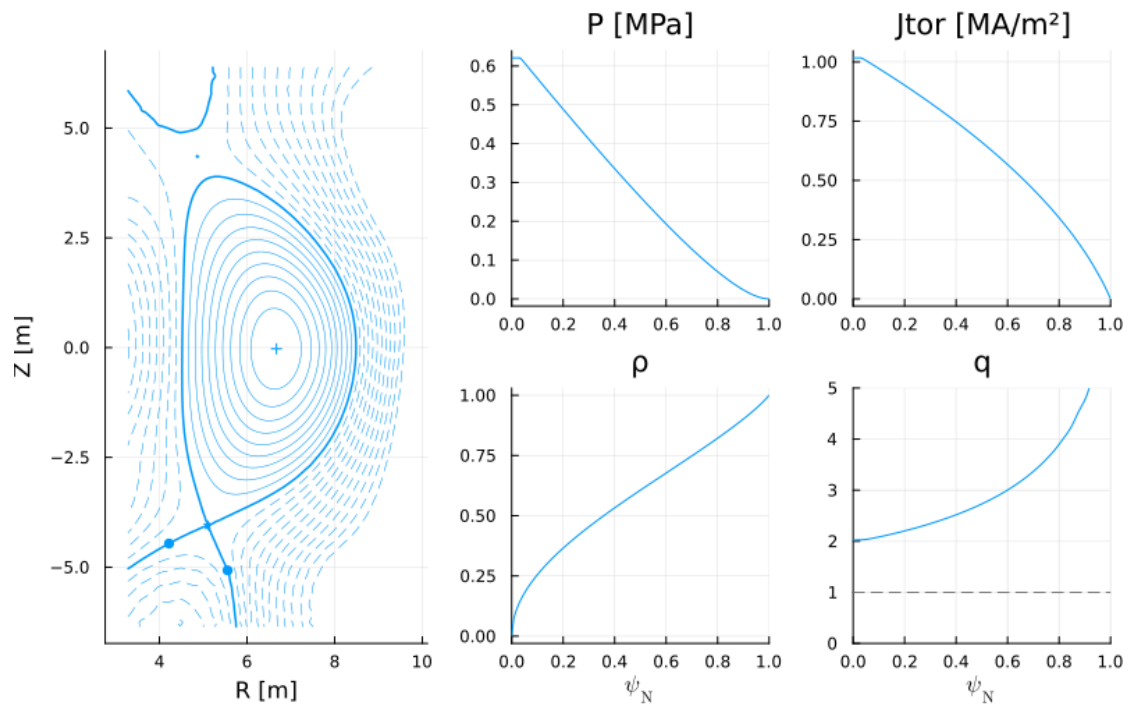
```
In [7]: plot(dd.build)
```

Out[7]:

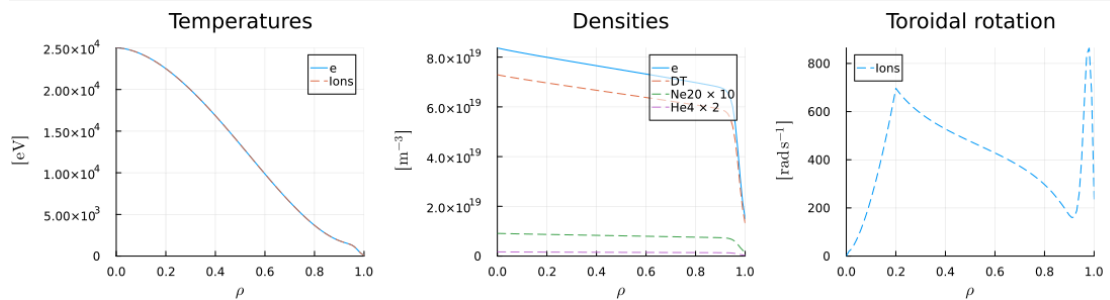


```
In [8]: plot(dd.equilibrium)
```

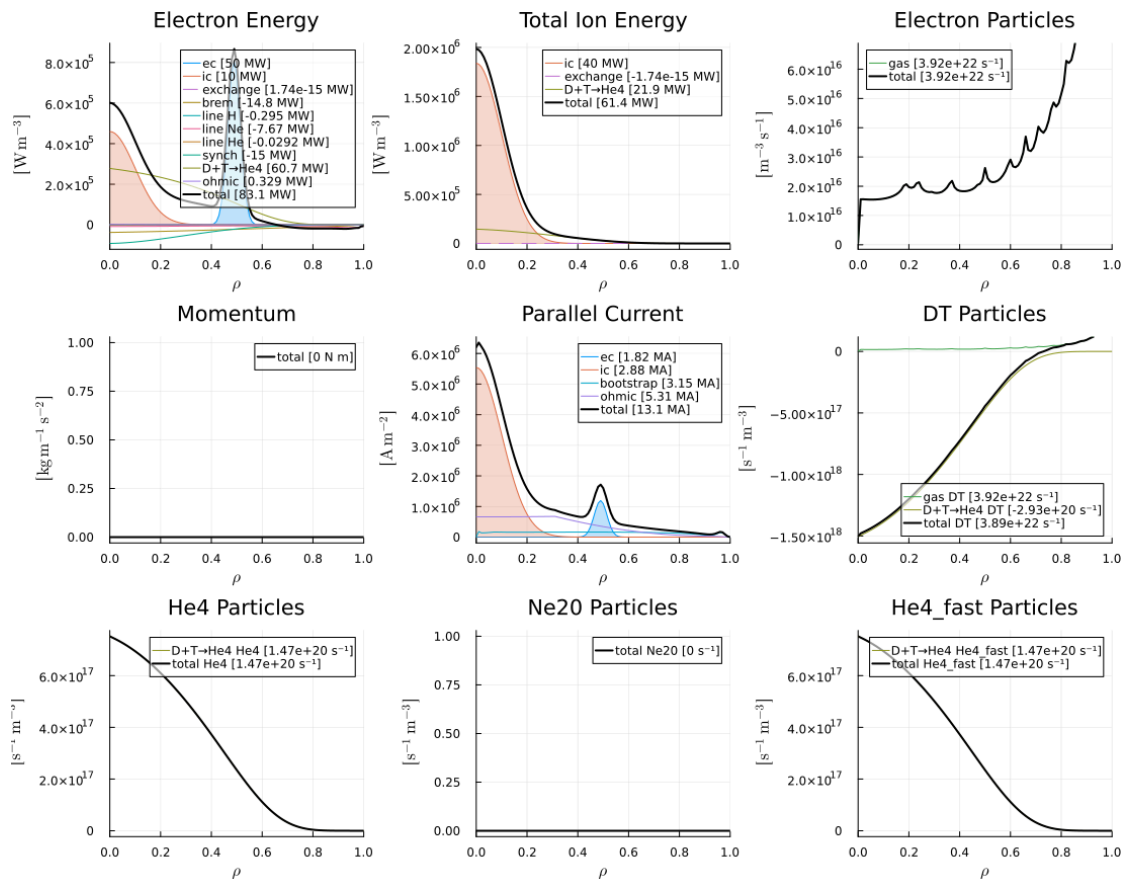
Out[8]:

In [9]: `plot(dd.core_profiles)`

Out[9]:

In [10]: `plot(dd.core_sources)`

Out[10]:



We can `@checkin` and `@checkout` variables with an associated tag.

This is handy to save and restore (checkpoint) our progress without having to always start from scratch (we'll use this later).

```
In [11]: @checkin :init dd ini act
```

Running Actors

Let's now run a series of actors and play around with plotting to get a sense of what each individual actor does.

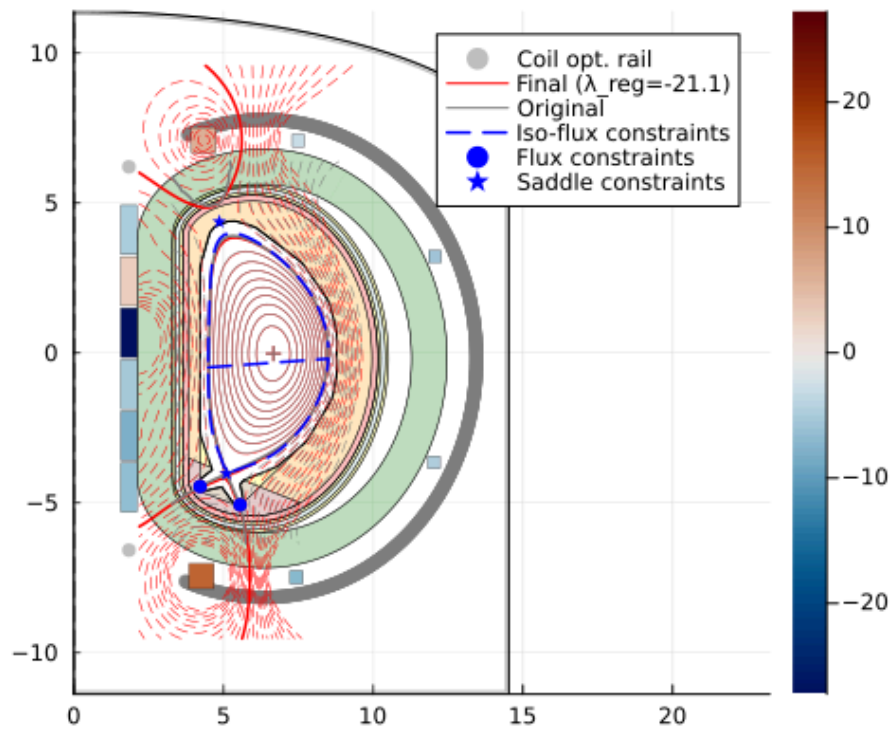
Here's how we can restore things back to after the initialization stage (in case we did anything else in between)

```
In [12]: @checkout :init dd ini act
```

Actors in FUSE can be executed by passing two arguments to them: `dd` and `act`.

Let's start by positioning the PF coils, so that we stand a chance to reproduce the desired plasma shape. This will be important to ensure the stability of the `ActorStationaryPlasma` that we are going to run next.

```
In [13]: FUSE.ActorPFdesign(dd, act; do_plot=true); # instead of setting `act.ActorPFdesi
actors: PFdesign
```



The `ActorStationaryPlasma` iterates between plasma transport, pedestal, equilibrium and sources to return a self-consistent plasma solution

```
In [14]: peq = plot(dd.equilibrium; label="before")
pcp = plot(dd.core_profiles; color=:gray, label="before")
#act.ActorFluxMatcher.verbose = true
act.ActorFluxMatcher.algorithm = :anderson
#act.ActorFluxMatcher.step_size = 0.1
FUSE.ActorStationaryPlasma(dd, act);
```

```

actors: StationaryPlasma
actors: ----- 1/5
actors: Sources
actors: SimpleEC
actors: SimpleIC
actors: NeutralFueling
actors: Pedestal
actors: EPED
actors: CoreTransport
actors: FluxMatcher
r Warning: Non-matching vector element type detected – allocating type conversion. For zero-allocation, pre-convert your data: `x_typed = Float64.(x)`
L @ FastInterpolations C:\Users\Croma\.julia\packages\FastInterpolations\zqwkS\src\core\utils.jl:49
actors: FluxCalculator
actors: TGLF
actors: Neoclassical
actors: Current
actors: QED
actors: SawteethSource
actors: Equilibrium
actors: TEQUILA
actors: ----- 1/5 @ 419.93%
actors: Sources
actors: SimpleEC
actors: SimpleIC
actors: NeutralFueling
actors: Pedestal
actors: EPED
actors: CoreTransport
actors: FluxMatcher
actors: FluxCalculator
actors: TGLF
actors: Neoclassical
actors: Current
actors: QED
actors: SawteethSource
actors: Equilibrium
actors: TEQUILA
actors: ----- 2/5 @ 544.04%
actors: Sources
actors: SimpleEC
actors: SimpleIC
actors: NeutralFueling
actors: Pedestal
actors: EPED
actors: CoreTransport
actors: FluxMatcher
actors: FluxCalculator
actors: TGLF
actors: Neoclassical
actors: Current
actors: QED
actors: SawteethSource
actors: Equilibrium
actors: TEQUILA
actors: ----- 3/5 @ 986.25%
actors: Sources
actors: SimpleEC
actors: SimpleIC

```

```

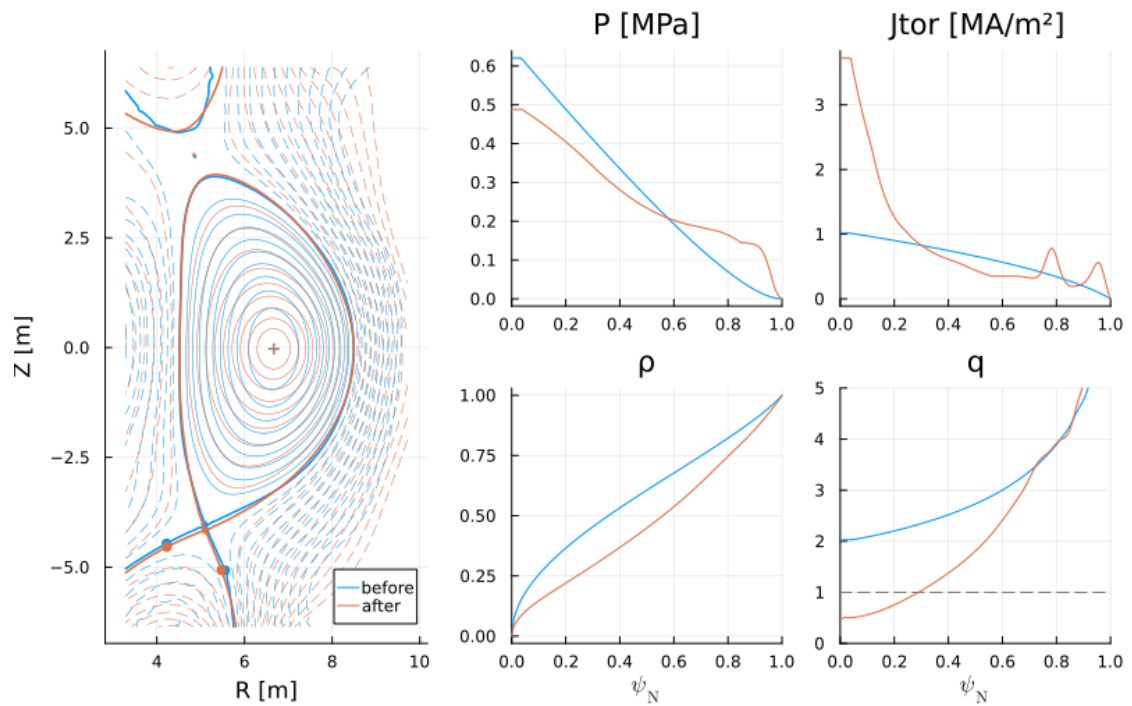
actors: NeutralFueling
actors: Pedestal
actors: EPED
actors: CoreTransport
actors: FluxMatcher
actors: FluxCalculator
actors: TGLF
actors: Neoclassical
actors: Current
actors: QED
actors: SawteethSource
actors: Equilibrium
actors: TEQUILA
actors: ----- 4/5 @ 763.63%
actors: Sources
actors: SimpleEC
actors: SimpleIC
actors: NeutralFueling
actors: Pedestal
actors: EPED
actors: CoreTransport
actors: FluxMatcher
actors: FluxCalculator
actors: TGLF
actors: Neoclassical
actors: Current
actors: QED
actors: SawteethSource
actors: Equilibrium
actors: TEQUILA
actors: ----- 5/5 @ 531.17%
⚠ Warning: Max number of iterations (5) has been reached with convergence error o
f (1)[0.21, 0.272, 0.493, 0.382, 0.266](5) compared to threshold of 0.05
↳ @ FUSE C:\Users\Croma\.julia\packages\FUSE\1R86C\src\actors\compound\stationary
_plasma_actor.jl:237

```

we can compare equilibrium before and after the self-consistency loop

```
In [15]: plot!(peq, dd.equilibrium; label="after")
```

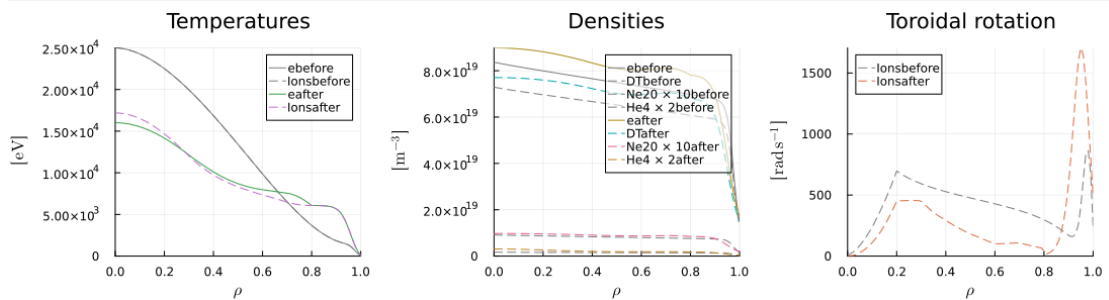

Out[15]:



we can compare core_profiles before and after the self-consistency loop

```
In [16]: plot!(pcp, dd.core_profiles; label="after")
```

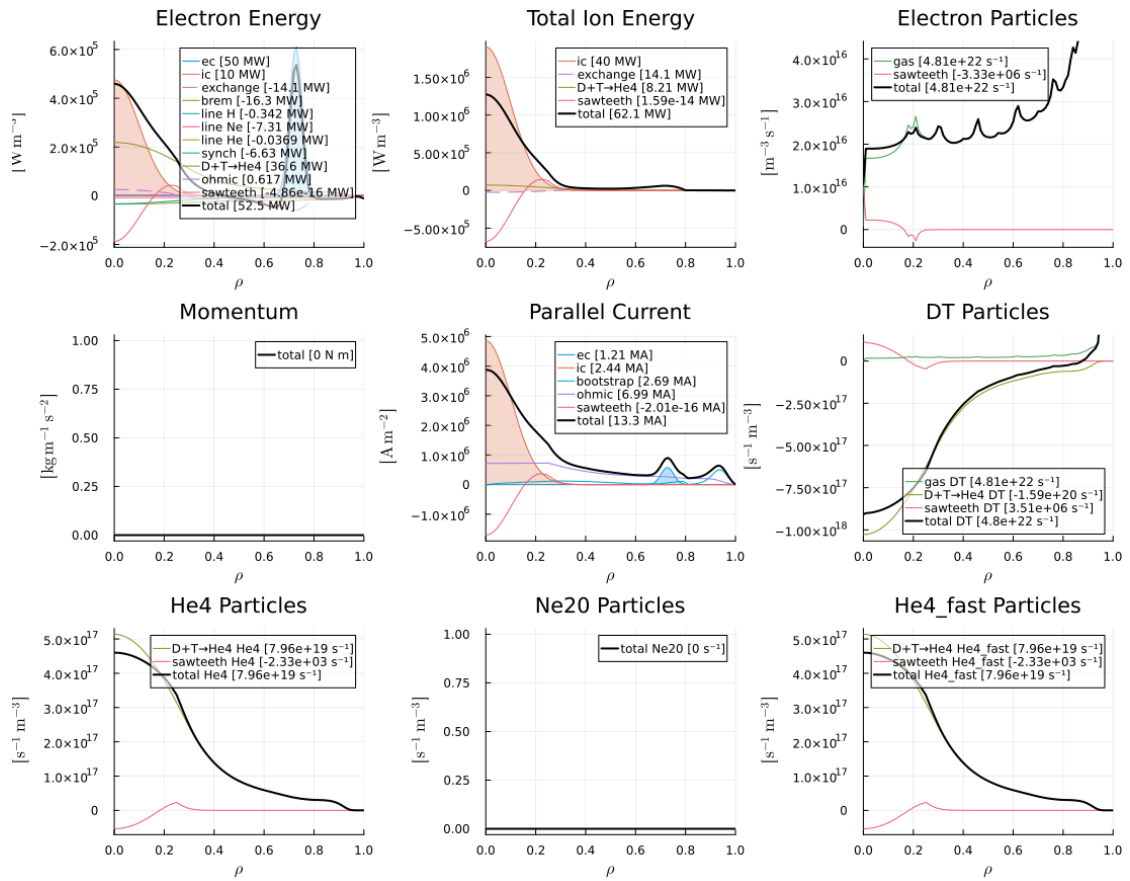
Out[16]:



here are the sources

```
In [17]: plot(dd.core_sources)
```

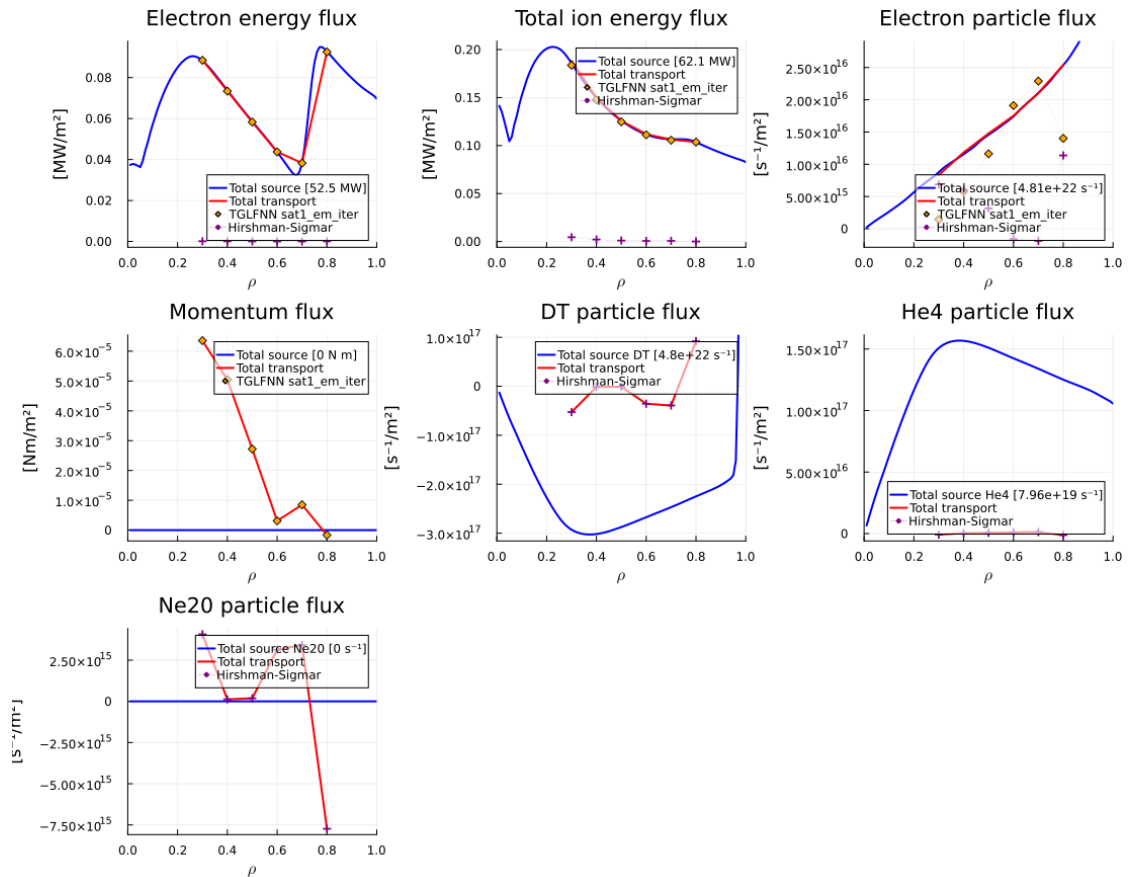
Out[17]:



and the flux-matched transport

In [18]: `plot(dd.core_transport)`

Out[18]:

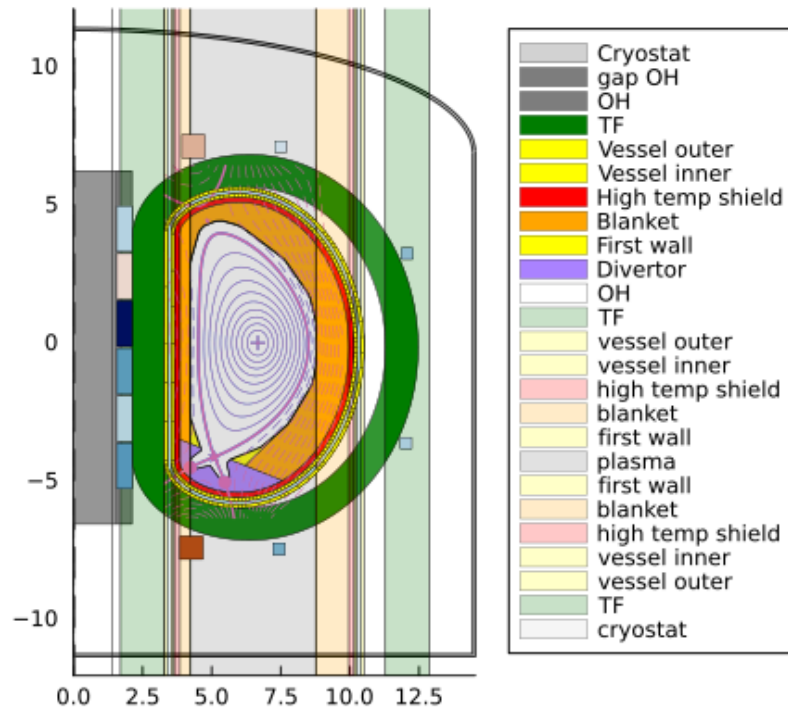


HFS sizing actor changes the thickness of the OH and TF layers on the high field side to satisfy current and stresses constraints

```
In [19]: plot(dd.build)
FUSE.ActorHFSsizing(dd, act);
plot!(dd.build; cx=false)
```

actors: HFSsizing
actors: FluxSwing
actors: Stresses

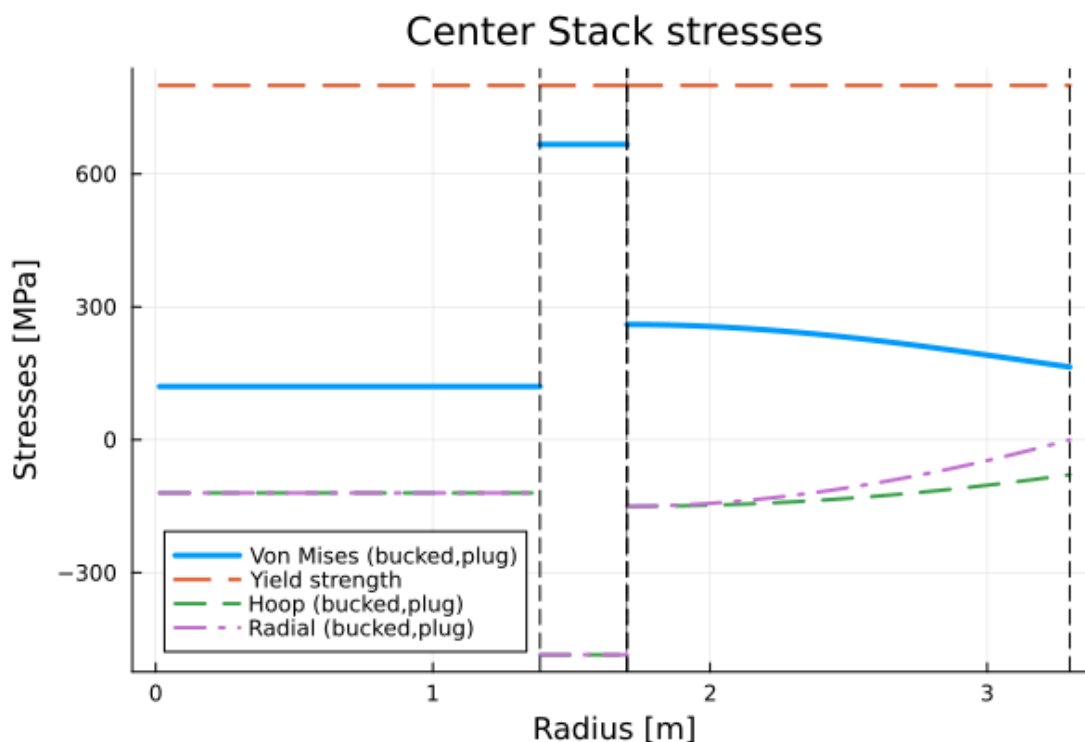
Out[19]:



The stresses on the center stack are stored in the `solid_mechanics` IDS

```
In [20]: plot(dd.solid_mechanics.center_stack.stress)
```

Out[20]:

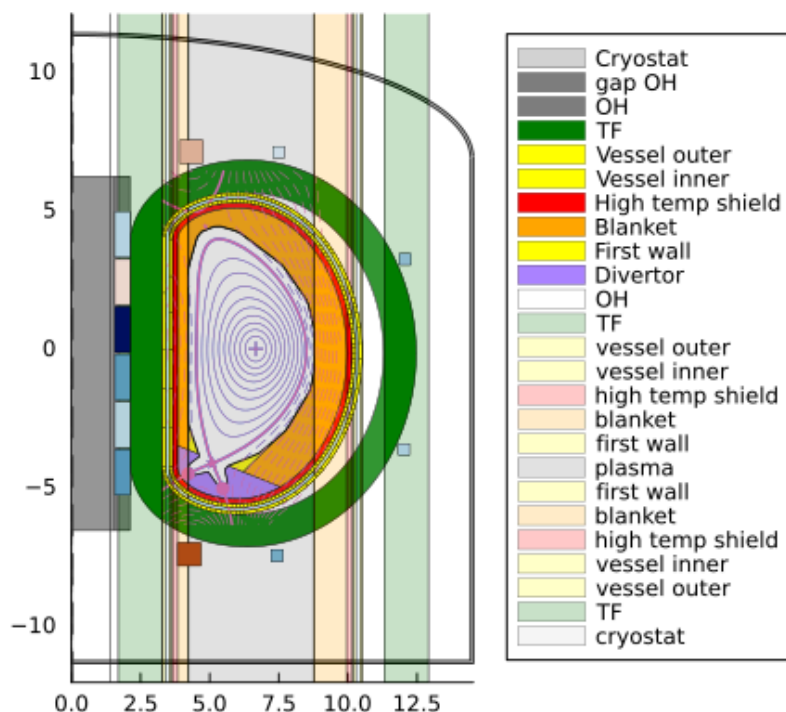


LFS sizing actors change location of the outer TF leg to meet ripple requirements

```
In [21]: plot(dd.build)
FUSE.ActorLFSSizing(dd, act);
plot!(dd.build; cx=false)
```

actors: LFSsizing

Out[21]:



A custom `show()` method is defined to print the summary of `dd.build.layer`

```
In [22]: dd.build.layer
```

Out[22]: 23x10 DataFrame

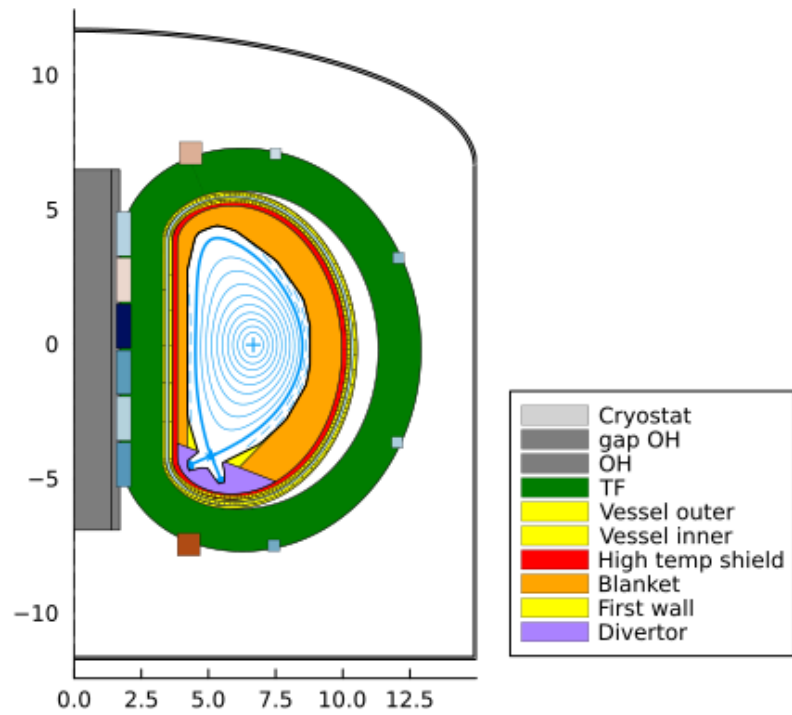
Row	group	details			type	ΔR	R_start
R_end	material	area	volume	shape			
Float64	String	String	Float64	Float64	String	Float64	Float64
	String			String			
1	in					1.38637	0.0
1.38637	steel	20.0722	98.9922	rectangle			
2	in			oh		0.314283	1.3863
7	1.70065	nb3sn	6.96307	80.5936	rectangle		
3	hfs			tf		1.59729	1.7006
5	3.29794	nb3sn_kdemo	42.9267	889.509	convex hull		
4	hfs	gap tf vacuum vessel				0.0	3.2979
4	3.29794	vacuum	7.46579	490.894	double ellipse		
5	hfs	vacuum outer			vessel	0.0982993	3.2979
4	3.39624	steel	3.06133	124.725	negative offset		
6	hfs	gap water				0.147449	3.3962
4	3.54369	water	4.47813	182.564	negative offset		
7	hfs	vacuum inner			vessel	0.0982993	3.5436
9	3.64199	steel	2.90951	118.694	negative offset		
8	hfs	gap high temp shield vacuum vess...				0.00982993	3.6419
9	3.65182	vacuum	0.287611	11.7367	negative offset		
9	hfs	high temp shield				0.196599	3.6518
2	3.84842	steel	5.62468	229.669	negative offset		
10	hfs			blanket		0.373538	3.8484
2	4.22196	lithium_lead	20.8601	951.677	negative offset		
11	hfs	first wall				0.0196599	4.2219
6	4.24162	tungsten	1.02384	34.0701	offset		
12	lhfs			plasma		4.51651	4.2416
2	8.75812	plasma	32.1361	1252.5			
13	lfs	first wall				0.0196599	8.7581
2	8.77778	tungsten	1.02384	34.0701	offset		
14	lfs			blanket		1.17959	8.7777
8	9.95738	lithium_lead	20.8601	951.677	negative offset		
15	lfs	high temp shield				0.196599	9.9573
8	10.154	steel	5.62468	229.669	negative offset		
16	lfs	gap high temp shield vacuum vess...				0.0442026	10.154
10.1982	vacuum	0.287611	11.7367	negative offset			
17	lfs	vacuum inner			vessel	0.0982993	10.1982
10.2965	steel	2.90951	118.694	negative offset			
18	lfs	gap water				0.147449	10.2965
10.4439	water	4.47813	182.564	negative offset			
19	lfs	vacuum outer			vessel	0.0982993	10.4439
10.5422	steel	3.06133	124.725	negative offset			
20	lfs	gap tf vacuum vessel				0.775582	10.5422
11.3178	vacuum	7.46579	490.894	double ellipse			
21	lfs			tf		1.59729	11.3178
12.9151	nb3sn_kdemo	42.9267	889.509	convex hull			
22	out					1.96599	0.0
14.8811	vacuum	164.179	8376.89				
23	out			cryostat		0.0982993	0.0
14.9794	steel	4.86736	309.729	silo			

ActorHFSizing and ActorLFSizing only change the layer's thicknesses, so we then need to trigger a build of the 2D cross-sections after them:

```
In [23]: FUSE.ActorCXbuild(dd, act);
          plot(dd.build)
```

actors: CXbuild

Out[23]:

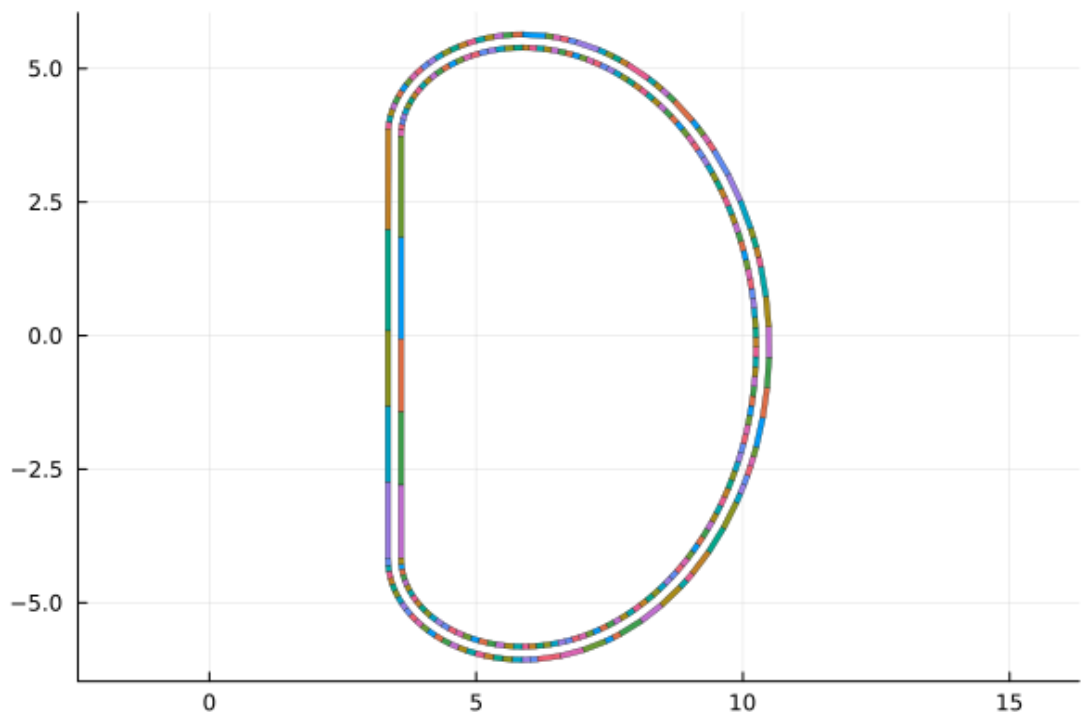


Generate passive structures information (for now the vacuum vessel)

```
In [24]: FUSE.ActorPassiveStructures(dd, act)
plot(dd.pf_passive)
```

actors: PassiveStructures

Out[24]:

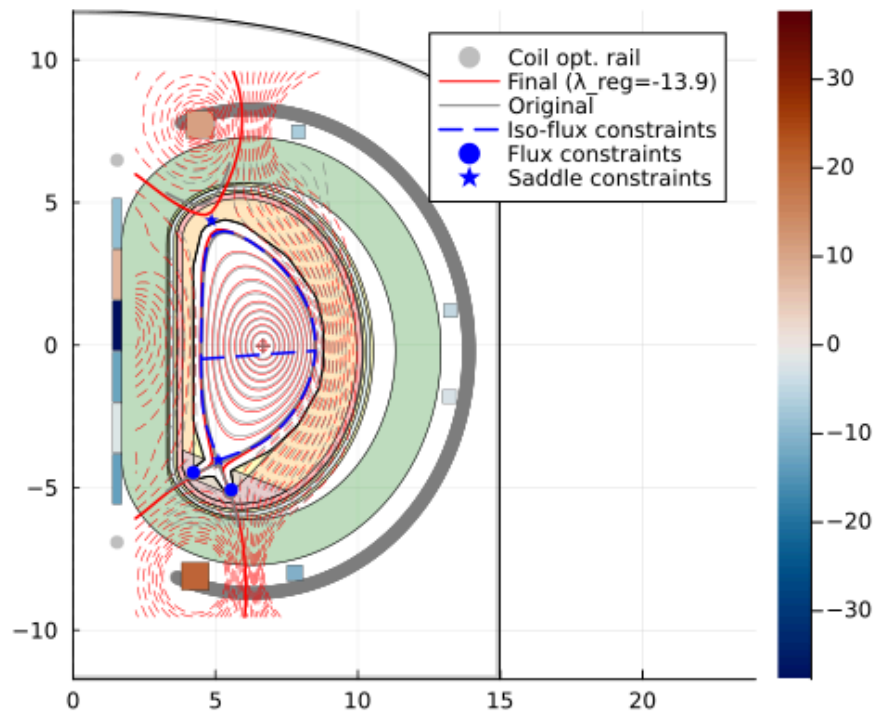


We can now give the PF coils their final position given the new build

```
In [25]: actor = FUSE.ActorPFdesign(dd, act);
plot(actor) # some actors define their own plot
```

actors: PFdesign

Out[25]:



With information about both pf_active and pf_passive we can now evaluate vertical stability

```
In [26]: FUSE.ActorVerticalStability(dd, act)
IMAS.freeze(dd.mhd_linear)
```

actors: VerticalStability

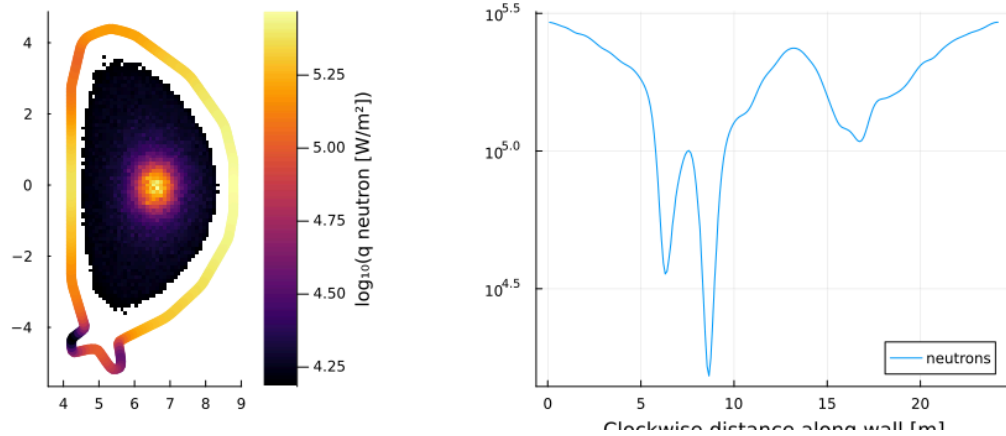
```
Out[26]: mhd_linear [DETACHED]
├ time → [0] [s]
└ time_slice
  └ 1
    └ time → 0 [s]
      └ toroidal_mode
        └ 1
          └ n_tor → 0
            └ perturbation_type
              └ description → "Vertical stability margin > 0.15 for stabilit
y"
                └ name → "m_s"
                  └ stability_metric → 0.143049
                    └ 2
                      └ n_tor → 0
                        └ perturbation_type
                          └ description → "Normalized vertical growth rate < 10 for stab
ility"
                            └ name → "γτ"
                              └ stability_metric → 8.58433
```

The `ActorNeutronics` calculates the heat flux on the first wall

```
In [27]: FUSE.ActorNeutronics(dd, act);
p = plot(; layout=2, size=(900, 350))
plot!(p, dd.neutronics.time_slice[].wall_loading, subplot=1)
plot!(p, FUSE.define_neutrons(dd, 100000)[1], dd.equilibrium.time_slice[]; subplot=2)
plot!(p, dd.neutronics.time_slice[].wall_loading; cx=false, subplot=2, ylabel="")
```

actors: Neutronics

Out[27]:



The **ActorBlanket** will change the thickness of the first wall, breeder, shield, and Li6 enrichment to achieve target TBR

```
In [28]: FUSE.ActorBlanket(dd, act);
print_tree(IMAS.freeze(dd.blanket); maxdepth=5)
```

actors: Blanket

blanket [DETACHED]

```
├─ module
│   └─ 1
│       └─ layer
│           ├── 1
│           │   ├── material → "tungsten"
│           │   ├── midplane_thickness → 0.0199566 [m]
│           │   └─ name → "lfs first wall"
│           ├── 2
│           │   ├── material → "lithium-lead: Li6/7=90.000%"
│           │   ├── midplane_thickness → 1.28724 [m]
│           │   └─ name → "lfs blanket"
│           └─ 3
│               ├── material → "steel"
│               ├── midplane_thickness → 0.0886523 [m]
│               └─ name → "lfs high temp shield"
│       └─ name → "blanket"
│       └─ time_slice
│           └─ 1
│               ├── peak_escape_flux → 142709 [W/m^2]
│               ├── peak_wall_flux → 774724 [W/m^2]
│               ├── power_incident_neutrons → 8.25451e+06 [W]
│               ├── power_incident_radiated → 0 [W]
│               ├── power_thermal_extracted → 9.90541e+06 [W]
│               ├── power_thermal_neutrons → 9.90541e+06 [W]
│               ├── power_thermal_radiated → 0 [W]
│               ├── time → 0 [s]
│               └─ tritium_breeding_ratio → 1.66152
└─ time → [0] [s]
└─ tritium_breeding_ratio → [0.0764439]
```

The **ActorDivertors** actor calculates the divertors heat flux

```
In [29]: FUSE.ActorDivertors(dd, act);
print_tree(IMAS.freeze(dd.divertors); maxdepth=4)
```

actors: Divertors


```

divertors [DETACHED]
├─ divertor
│   └─ 1
│       ├── power_black_body
│       │   ├── data → [0] [W]
│       │   └─ time → [0] [s]
│       ├── power_conducted
│       │   ├── data → [1.14861e+08] [W]
│       │   └─ time → [0] [s]
│       ├── power_convected
│       │   ├── data → [0] [W]
│       │   └─ time → [0] [s]
│       ├── power_currents
│       │   ├── data → [0] [W]
│       │   └─ time → [0] [s]
│       ├── power_incident
│       │   ├── data → [2515.67] [W]
│       │   └─ time → [0] [s]
│       ├── power_neutrals
│       │   ├── data → [0] [W]
│       │   └─ time → [0] [s]
│       ├── power_radiated
│       │   ├── data → [0] [W]
│       │   └─ time → [0] [s]
│       ├── power_recombination_neutrals
│       │   ├── data → [0] [W]
│       │   └─ time → [0] [s]
│       ├── power_recombination_plasma
│       │   ├── data → [0] [W]
│       │   └─ time → [0] [s]
│       ├── power_thermal_extracted
│       │   ├── data → [2515.67] [W]
│       │   └─ time → [0] [s]
│       └─ target
│           ├── 1
│           │   :
│           └─ 2
│               :
└─ time → [0] [s]

```

The `ActorBalanceOfPlant` calculates the optimal cooling flow rates for the heat sources (breeder, divertor, and wall) and get an efficiency for the electricity conversion cycle

```
In [30]: FUSE.ActorBalanceOfPlant(dd, act);
         IMAS.freeze(dd.balance_of_plant)
```

```

actors: BalanceOfPlant
actors: ThermalPlant
actors: PowerNeeds

```

```

Out[30]: balance_of_plant [DETACHED]
├─ Q_plant → [0.0539961]
├─ power_electric_net → [-1.37171e+08] [W]
├─ power_electric_plant_operation
│   └─ system
│       ├── 1
│       │   ├── index → 1
│       │   ├── name → "HCD"
│       │   ├── power → [1e+08] [W]
│       │   └─ subsystem
│       │       ├── 1
│       │       │   ├── index → 1
│       │       │   ├── name → "nbi"
│       │       │   └─ power → [0] [W]
│       │       ├── 2
│       │       │   ├── index → 2
│       │       │   ├── name → "ec_launchers"
│       │       │   └─ power → [5e+07] [W]
│       │       ├── 3
│       │       │   ├── index → 3
│       │       │   ├── name → "ic_antennas"
│       │       │   └─ power → [5e+07] [W]
│       │       └─ 4
│       │           ├── index → 4
│       │           ├── name → "lh_antennas"
│       │           └─ power → [0] [W]
│       ├── 2
│       │   ├── index → 3
│       │   ├── name → "cryostat"
│       │   └─ power → [3e+07] [W]
│       ├── 3
│       │   ├── index → 4
│       │   ├── name → "tritium_handling"
│       │   └─ power → [1.5e+07] [W]
│       └─ 4
│           ├── index → 6
│           ├── name → "pf_active"
│           └─ power → [0] [W]
│   └─ total_power → [1.45e+08] [W]
├─ power_plant
│   ├── heat_load
│   │   ├── breeder → [9.90541e+06] [W]
│   │   ├── divertor → [2515.67] [W]
│   │   └─ wall → [3.06088e+07] [W]
│   ├── power_cycle_type → "rankine"
│   ├── power_electric_generated → [7.82944e+06] [W]
│   ├── total_heat_supplied → [4.05167e+07] [W]
│   ├── thermal_efficiency_plant → [0.19324]
│   └─ time → [0] [s]

```

ActorCosting will break down the capital and operational costs

```

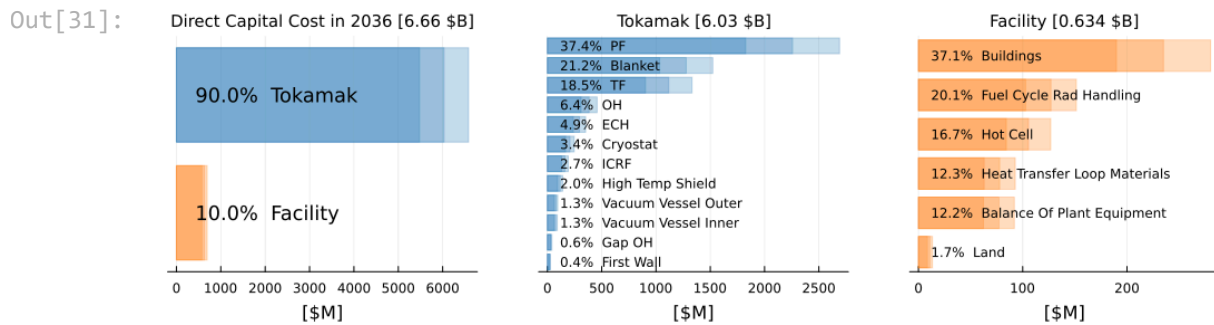
In [31]: FUSE.ActorCosting(dd, act)
         plot(dd.costing)

```

```

actors: Costing
actors: CostingARIES
Warning: Inflation data not available for 2026
@ FUSE C:\Users\Croma\.julia\packages\FUSE\lR86C\src\actors\costing\costing_util.jl:115

```



Let's checkpoint our results

```
In [32]: @checkpoint :manual dd ini act
```

Whole facility design

Here we restore the `:init` checkpoint that we had previously stored. Resetting any changes to `dd`, `ini`, and `act` that we did in the meantime.

```
In [33]: @checkout :init dd ini act
```

Actors can call other actors, creating workflows. For example, the `ActorWholeFacility` can be used to get a self-consistent stationary whole facility design.

```
In [34]: FUSE.ActorWholeFacility(dd, act);
```

```

actors: WholeFacility
actors: PFdesign
actors: PFdesign
actors: StationaryPlasma
actors: ----- 1/5
actors: Sources
actors:   SimpleEC
actors:   SimpleIC
actors:   NeutralFueling
actors: Pedestal
actors: EPED
actors: CoreTransport
actors:   FluxMatcher
actors:   FluxCalculator
actors:   TGLF
actors:   Neoclassical
actors: Current
actors: QED
actors: SawteethSource
actors: Equilibrium
actors: TEQUILA
actors: ----- 1/5 @ 420.06%
actors: Sources
actors:   SimpleEC
actors:   SimpleIC
actors:   NeutralFueling
actors: Pedestal
actors: EPED
actors: CoreTransport
actors:   FluxMatcher
actors:   FluxCalculator
actors:   TGLF
actors:   Neoclassical
actors: Current
actors: QED
actors: SawteethSource
actors: Equilibrium
actors: TEQUILA
actors: ----- 2/5 @ 544.02%
actors: Sources
actors:   SimpleEC
actors:   SimpleIC
actors:   NeutralFueling
actors: Pedestal
actors: EPED
actors: CoreTransport
actors:   FluxMatcher
actors:   FluxCalculator
actors:   TGLF
actors:   Neoclassical
actors: Current
actors: QED
actors: SawteethSource
actors: Equilibrium
actors: TEQUILA
actors: ----- 3/5 @ 995.93%
actors: Sources
actors:   SimpleEC
actors:   SimpleIC
actors:   NeutralFueling

```

```

actors: Pedestal
actors: EPED
actors: CoreTransport
actors: FluxMatcher
actors: FluxCalculator
actors: TGLF
actors: Neoclassical
actors: Current
actors: QED
actors: SawteethSource
actors: Equilibrium
actors: TEQUILA
actors: ----- 4/5 @ 767.03%
actors: Sources
actors: SimpleEC
actors: SimpleIC
actors: NeutralFueling
actors: Pedestal
actors: EPED
actors: CoreTransport
actors: FluxMatcher
actors: FluxCalculator
actors: TGLF
actors: Neoclassical
actors: Current
actors: QED
actors: SawteethSource
actors: Equilibrium
actors: TEQUILA
actors: ----- 5/5 @ 385.03%
⚠ Warning: Max number of iterations (5) has been reached with convergence error o
f (1)[0.21, 0.272, 0.498, 0.384, 0.193](5) compared to threshold of 0.05
└ @ FUSE C:\Users\Croma\.julia\packages\FUSE\1R86C\src\actors\compound\stationary
_plasma_actor.jl:237
actors: HFSsizing
actors: FluxSwing
actors: Stresses
actors: LFSsizing
actors: CXbuild
actors: PFdesign
actors: Equilibrium
actors: TEQUILA
actors: CXbuild
actors: Neutronics
actors: Blanket
actors: CXbuild
actors: PassiveStructures
actors: Divertors
actors: PlasmaLimits
actors: VerticalStability
  Downloading artifact: onnxruntime_cpu
actors: TroyonBetaNN
actors: BalanceOfPlant
actors: ThermalPlant
actors: PowerNeeds
actors: Costing
actors: CostingARIES
⚠ Warning: Inflation data not available for 2026
└ @ FUSE C:\Users\Croma\.julia\packages\FUSE\1R86C\src\actors\costing\costing_uti
ls.jl:115

```

Let's check what we got at a glance with the `FUSE.digest(dd)` function:

```
In [35]: FUSE.digest(dd)
```

GEOMETRY EMPERATURES

R0 → 6.5 [m]
e0 → 17.5 [keV]
a → 2.01 [m]
i0 → 17.9 [keV]
1/ε → 3.24
Te> → 8.19 [keV]
κ → 2
Ti> → 7.5 [keV]
δ → 0.588
e0/<Te> → 2.13
ζ → -0.0136
i0/<Ti> → 2.38
Volume → 924 [m³]
Surface → 753 [m²]

DENSITIES RANSPORT

ne0 → 9.1e+19 [m⁻³]
e → 2.33 [s]
ne_ped → 6.68e+19 [m⁻³]
e_exp → 1.81 [s]
ne_line → 8.28e+19 [m⁻³]
98y2 → 0.83
<ne> → 7.57e+19 [m⁻³]
98y2_exp → 0.767
ne0/<ne> → 1.2
ds03 → 0.619
fGW → 0.827
ds03_exp → 0.552
zeff_ped → 2
α_thermalization → 0.84 [s]
<zeff> → 2
α_slowing_down → 0.901 [s]
impurities → DT Ne20 He4

SOURCES URRENTS

Pec → 50 [MW]
p_bs_aux_ohm → 12.9 [MA]
rho0_ec → 0.57 [MW]
p_ni → 7 [MA]
Pnbi → NaN [MW]
p_bs → 3.18 [MA]
Enbi1 → NaN [MeV]
p_aux → 3.82 [MA]
Pic → 50 [MW]
p_ohm → 5.95 [MA]
Plh → NaN [MW]
jima → 0.4
Paux_tot → 100 [MW]
lattop → 0.7 [Hours]
Pa → 47.9 [MW]

EQUILIBRIUM

B0 → 7.8 [T]
ip → 12.7 [MA]
q95 → 6.13
<Bpol> → 0.798 [T]
βpol_MHD → 0.757
βtor_MHD → 0.00834
βn_MHD → 1.02

PRESSURES

P0 → 0.554 [MPa]
<P> → 0.202 [MPa]
P0/<P> → 2.75
βn → 1.03
βn_th → 0.991

EXHAUST

Psol → 115 [MW]
PLH → 134 [MW]
Bpol_omp → 1.11 [T]
λq → 0.974 [mm]
qp01 → 2.22e+03 [MW/m²]
qpar → 1.21e+04 [MW/m²]
P/R0 → 17.7 [MW/m]
PB/R0 → 138 [MW T/m]

T

T

T

<

<

T

T

T

τ

τ

H

H

H

H

τ

τ

C

i

i

i

i

i

e

f

Pohm → 0.512 [MW]
Pheat → 148 [MW]
Prad_tot → -33.3 [MW]

PBp/R0 → 14.1 [MW T/m]
PBe/R0q95 → 6.95 [MW T/m]
neutrons_peak → 0.306 [MW/m²]

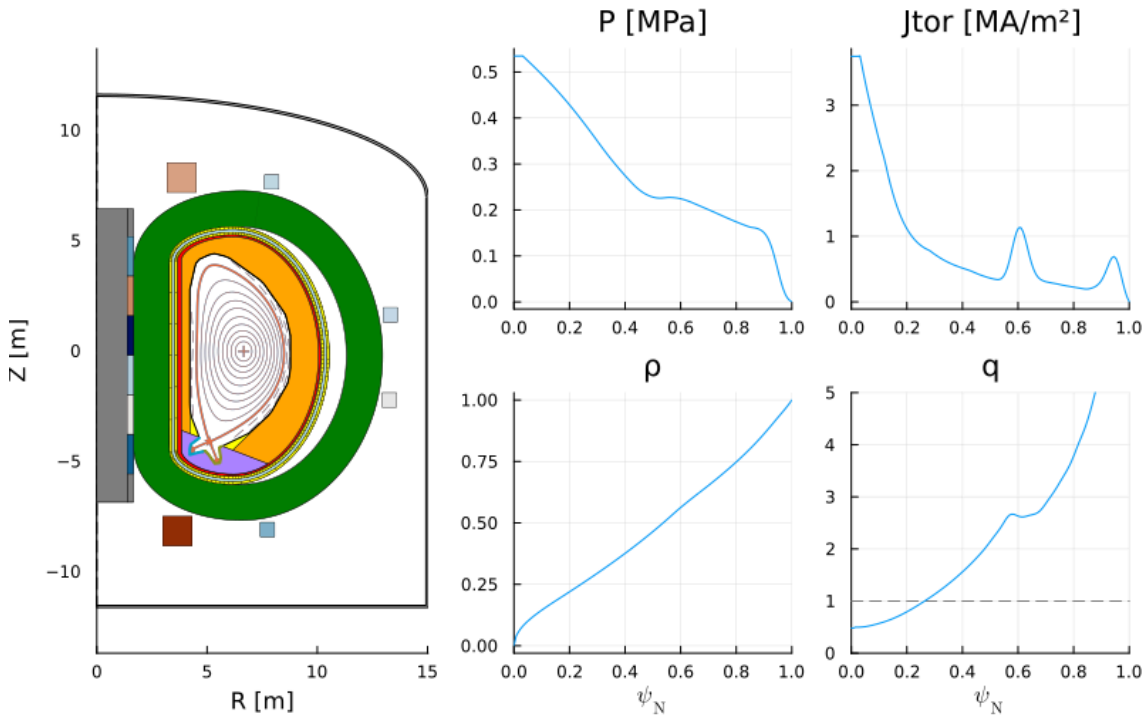
BOP
OSTING

BUILD C

Pfusion → 239 [MW]
apital_cost → 7.71 [\$B]
Qfusion → 2.39
evelized_CoE → Inf [\$/kWh]
thermal_cycle_type → rankine
F_of_total → 17.4 [%]
thermal_efficiency_plant → 19.4 [%]
OP_of_total → 1.04 [%]
thermal_efficiency_cycle → NaN [%]
lanket_of_total → 18.3 [%]
power_electric_generated → 8.32 [MW]
ryostat_of_total → 2.69 [%]
Pelectric_net → -137 [MW]
Qplant → 0.0574
TBR → 0.0685

PF_material → nb3sn c
TF_material → nb3sn_kdemo l
OH_material → nb3sn T
TF_max_b → 15.4 [T] B
OH_max_b → 14.8 [T] b
TF_j_margin → 6.14 c
OH_j_margin → 1.4
TF_stress_margin → 2.95
OH_stress_margin → 1.27

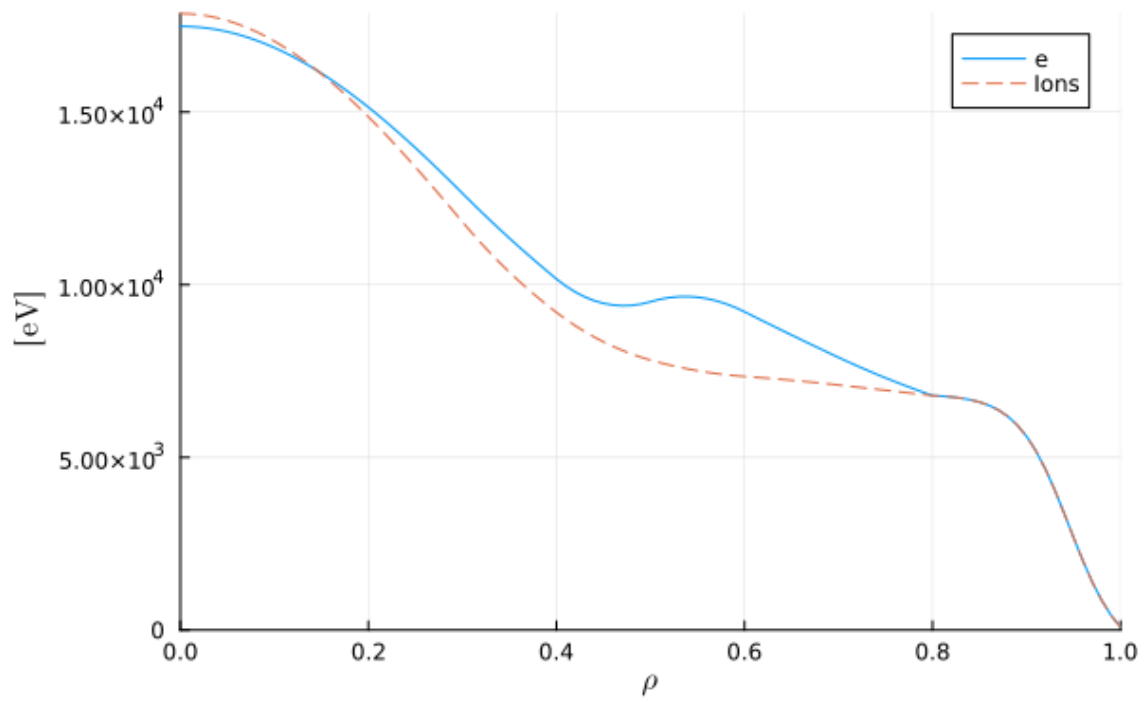
@ time = 0.0 [s]



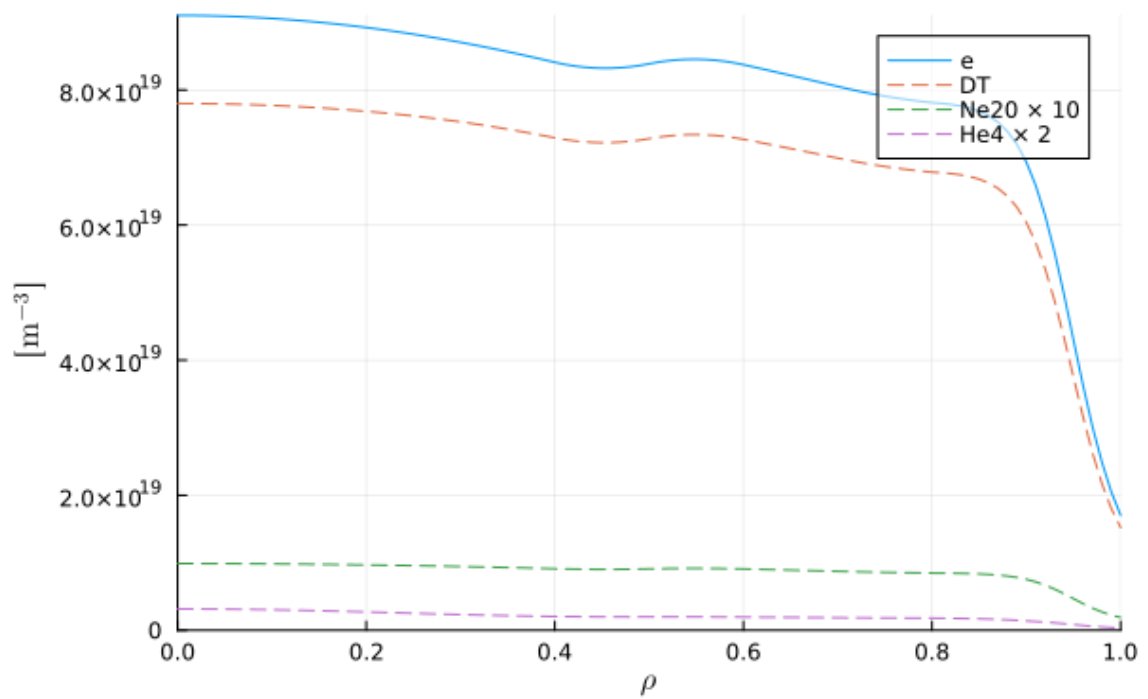
23x10 DataFrame

Row	group	details			type	ΔR	R_start
R_end	material	area	volume	shape	String	Float64	Float64
Float64	String	Float64	Float64	String			
1	in					1.38838	0.0
1.38838	steel	18.4649	80.5389	rectangle			
2	in			oh		0.267154	1.38838
1.65554	nb3sn	3.55303	33.9767	rectangle			
3	hfs			tf		1.64241	1.65554
3.29794	nb3sn_kdemo	62.4071	1293.17	convex hull			
4	hfs	gap tf	vacuum vessel			0.0	3.29794
3.29794	vacuum	7.52314	497.067	double ellipse			
5	hfs	vacuum	outer	vessel		0.0982993	3.29794
3.39624	steel	3.0898	126.315	negative offset			
6	hfs	gap water				0.147449	3.39624
3.54369	water	4.52083	184.937	negative offset			
7	hfs	vacuum	inner	vessel		0.0982993	3.54369
3.64199	steel	2.93797	120.268	negative offset			
8	hfs	gap high temp shield	vacuum vess...			0.00982993	3.64199
3.65182	vacuum	0.607407	31.2805	negative offset			
9	hfs	high temp		shield		0.196599	3.65182
3.84842	steel	3.84481	140.127	negative offset			
10	hfs			blanket		0.373538	3.84842
4.22196	lithium_lead	23.2776	1071.7	negative offset			
11	hfs	first		wall		0.0196599	4.22196
4.24162	tungsten	0.980479	32.0663	offset			
12	lhfs			plasma		4.51651	4.24162
8.75812	plasma	32.2579	1256.89				
13	lfs	first		wall		0.0199664	8.75812
8.77809	tungsten	0.977276	31.9838	offset			
14	lfs			blanket		1.27329	8.77809
10.0514	lithium_lead	23.2808	1071.78	negative offset			
15	lfs	high temp		shield		0.102599	10.0514
10.154	steel	3.84481	140.127	negative offset			
16	lfs	gap high temp shield	vacuum vess...			0.0442026	10.154
10.1982	vacuum	0.607407	31.2805	negative offset			
17	lfs	vacuum	inner	vessel		0.0982993	10.1982
10.2965	steel	2.93797	120.268	negative offset			
18	lfs	gap water				0.147449	10.2965
10.4439	water	4.52083	184.937	negative offset			
19	lfs	vacuum	outer	vessel		0.0982993	10.4439
10.5422	steel	3.0898	126.315	negative offset			
20	lfs	gap tf	vacuum vessel			0.775582	10.5422
11.3178	vacuum	7.52314	497.067	double ellipse			
21	lfs			tf		1.64241	11.3178
12.9602	nb3sn_kdemo	62.4071	1293.17	convex hull			
22	out					1.96599	0.0
14.9262	vacuum	166.102	8803.51				
23	out			cryostat		0.0982993	0.0
15.0245	steel	5.00508	328.552	silos			

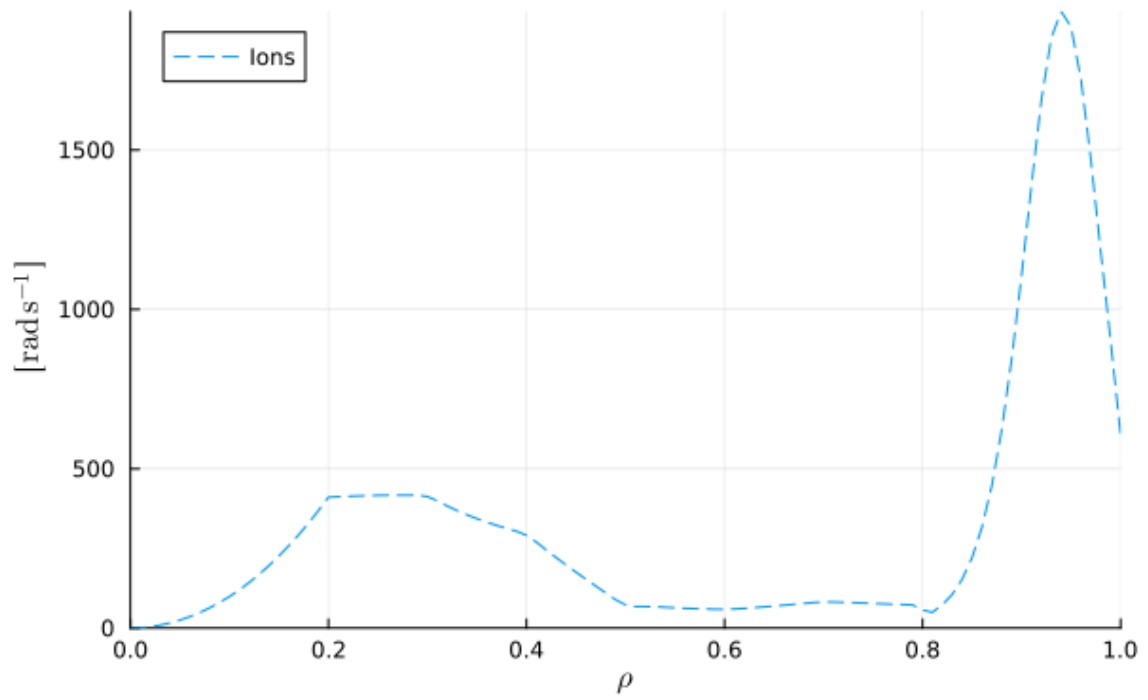
Temperatures



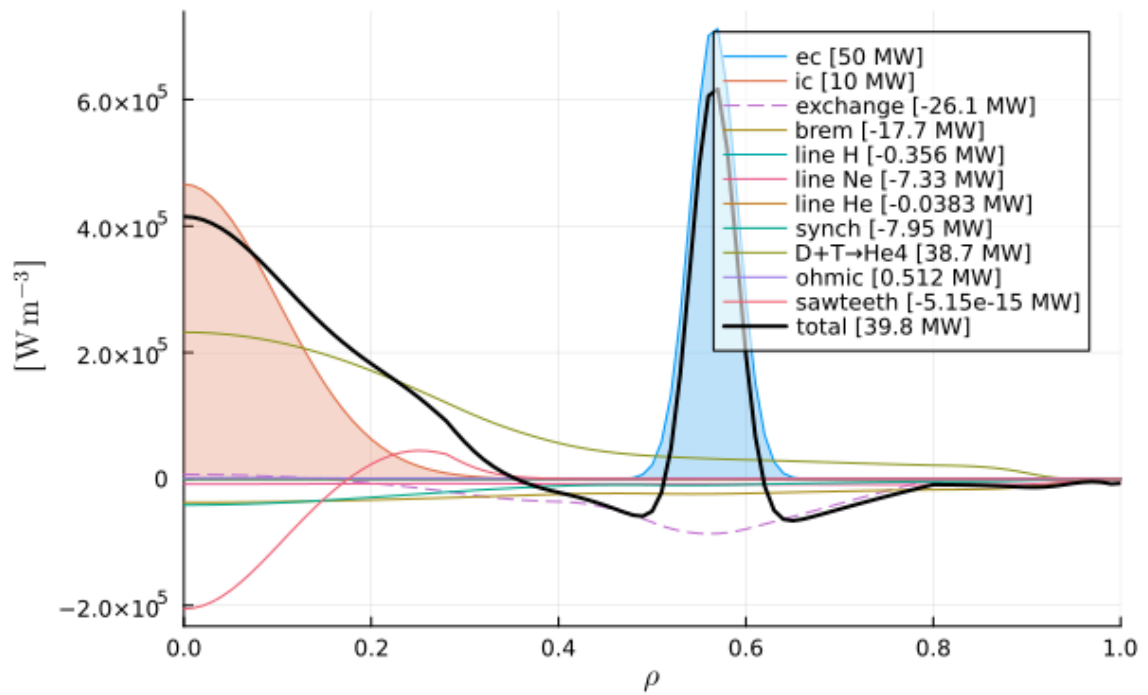
Densities



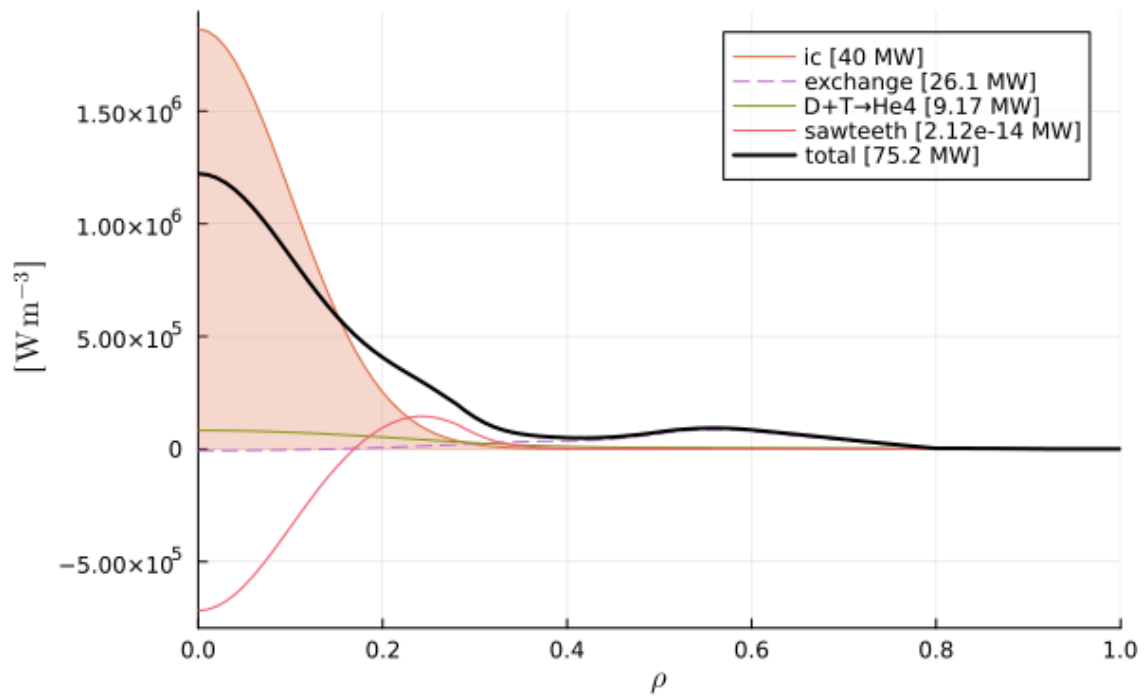
Toroidal rotation



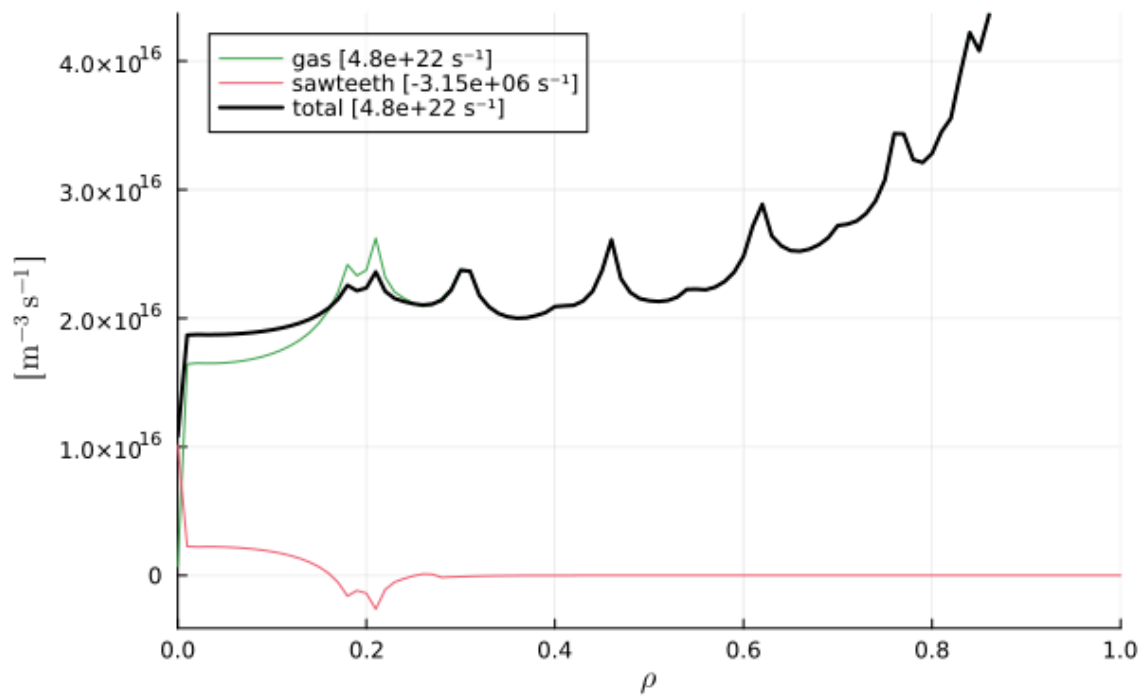
Electron Energy



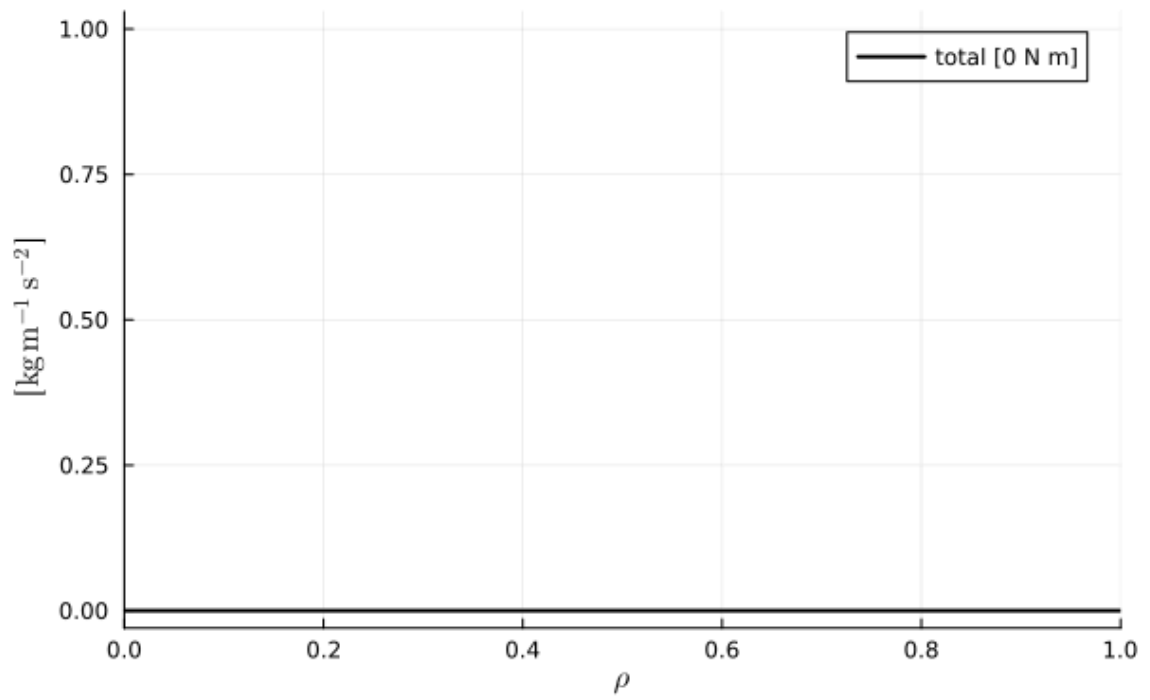
Total Ion Energy



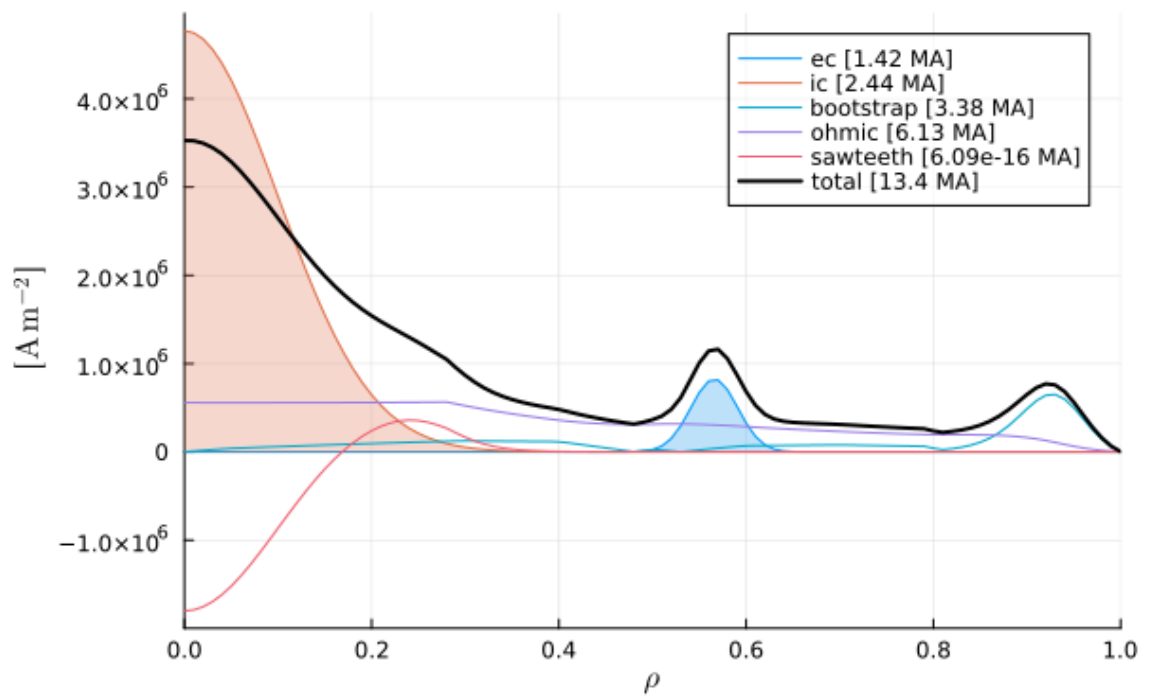
Electron Particles



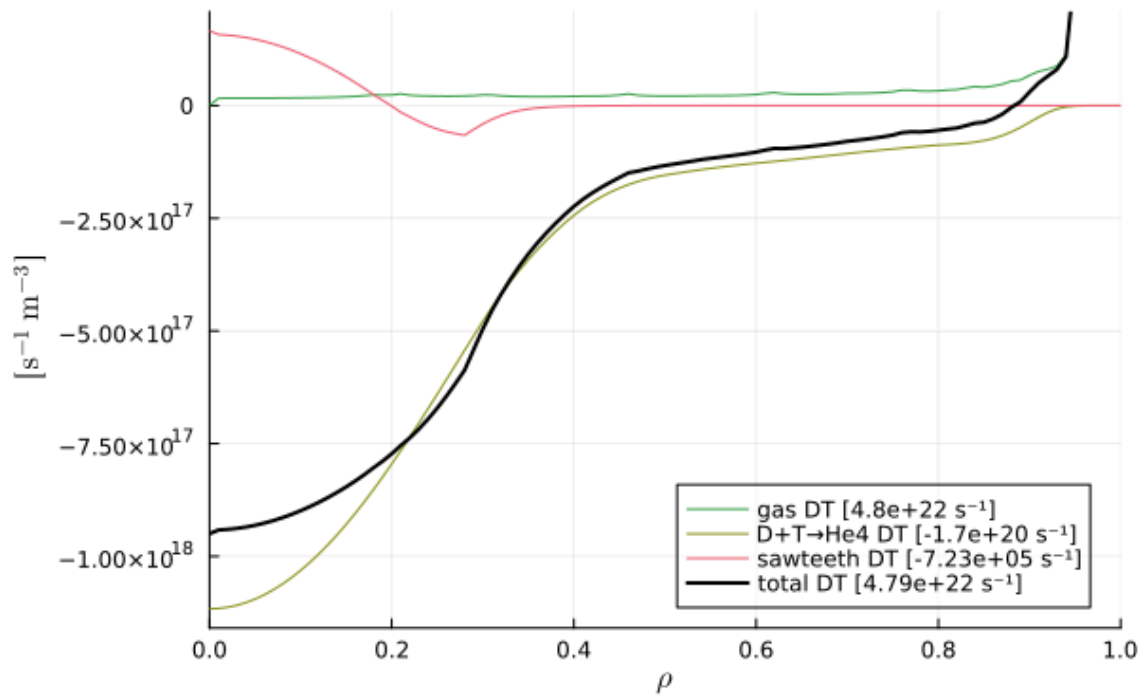
Momentum



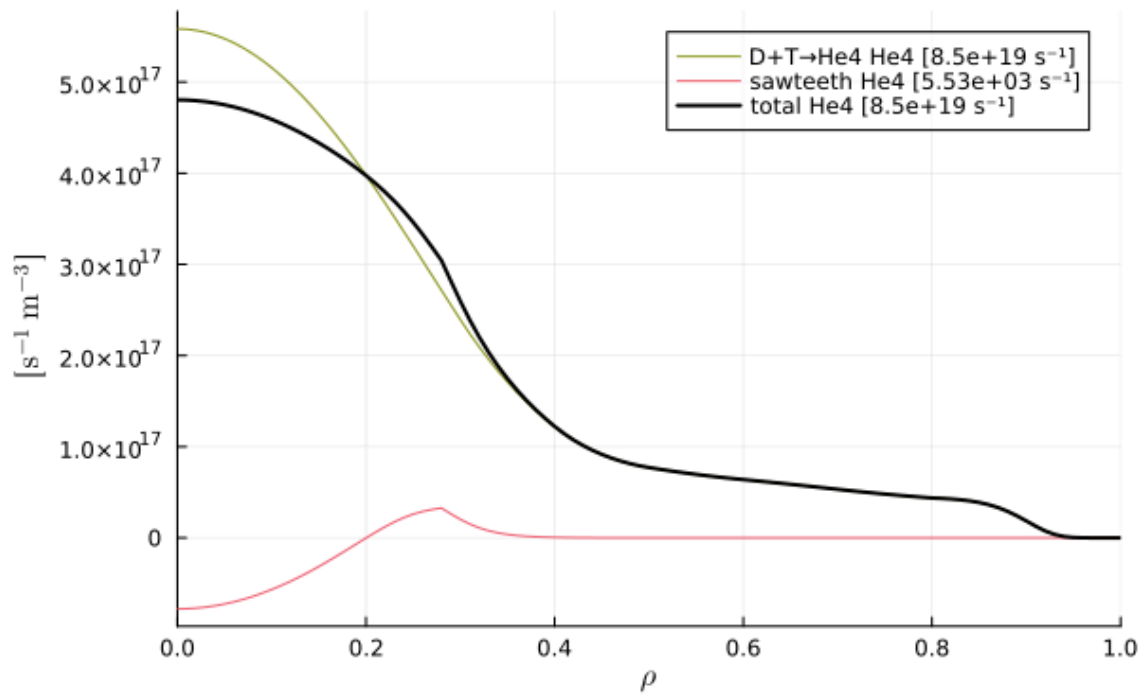
Parallel Current



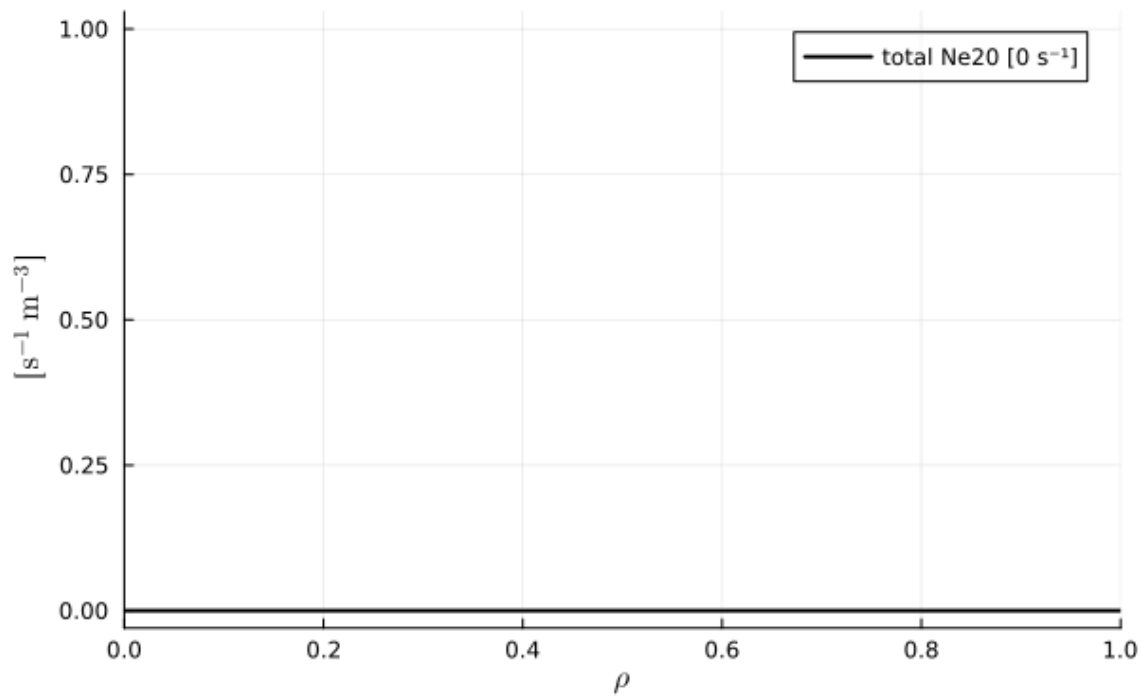
DT Particles



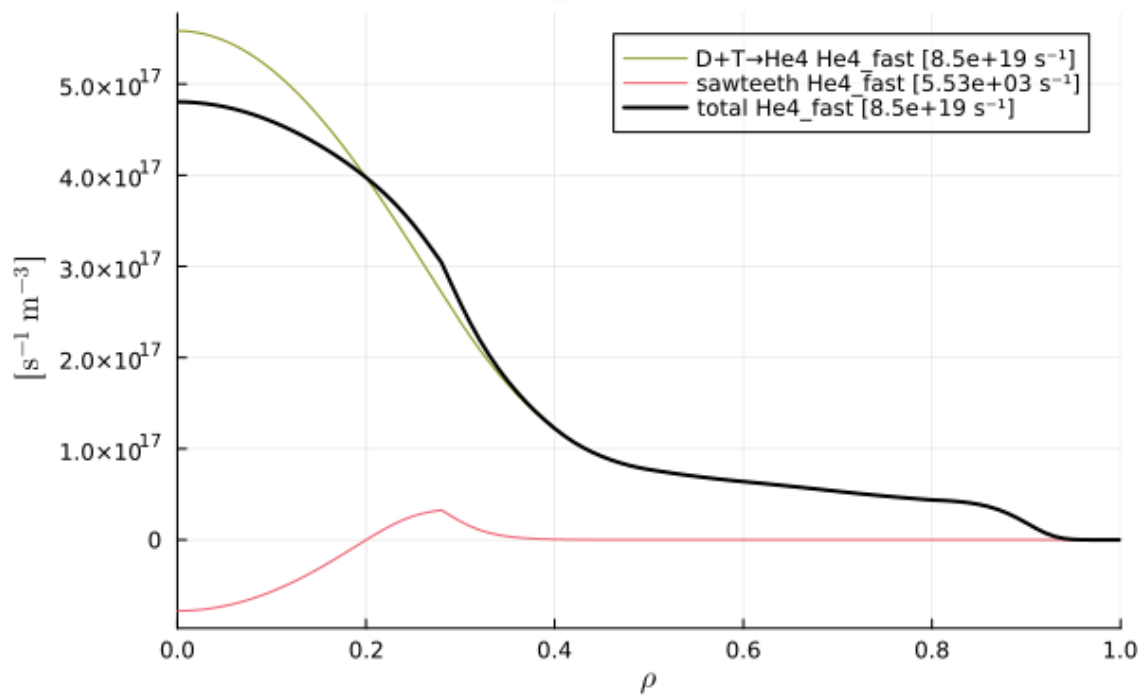
He4 Particles



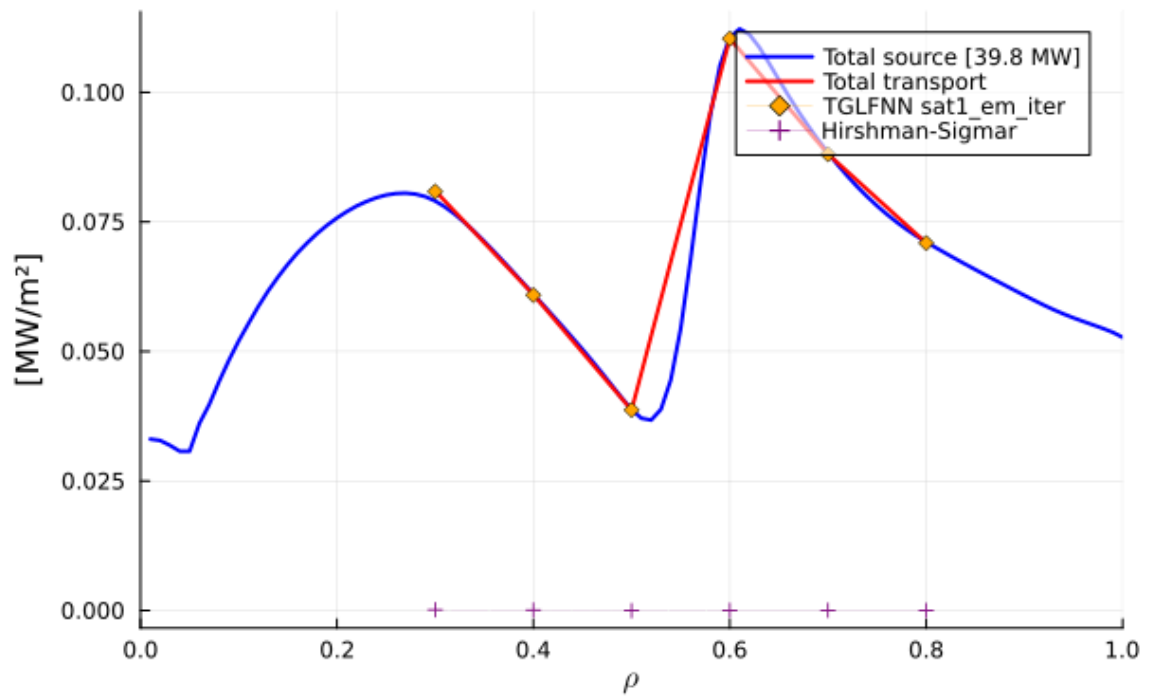
Ne20 Particles



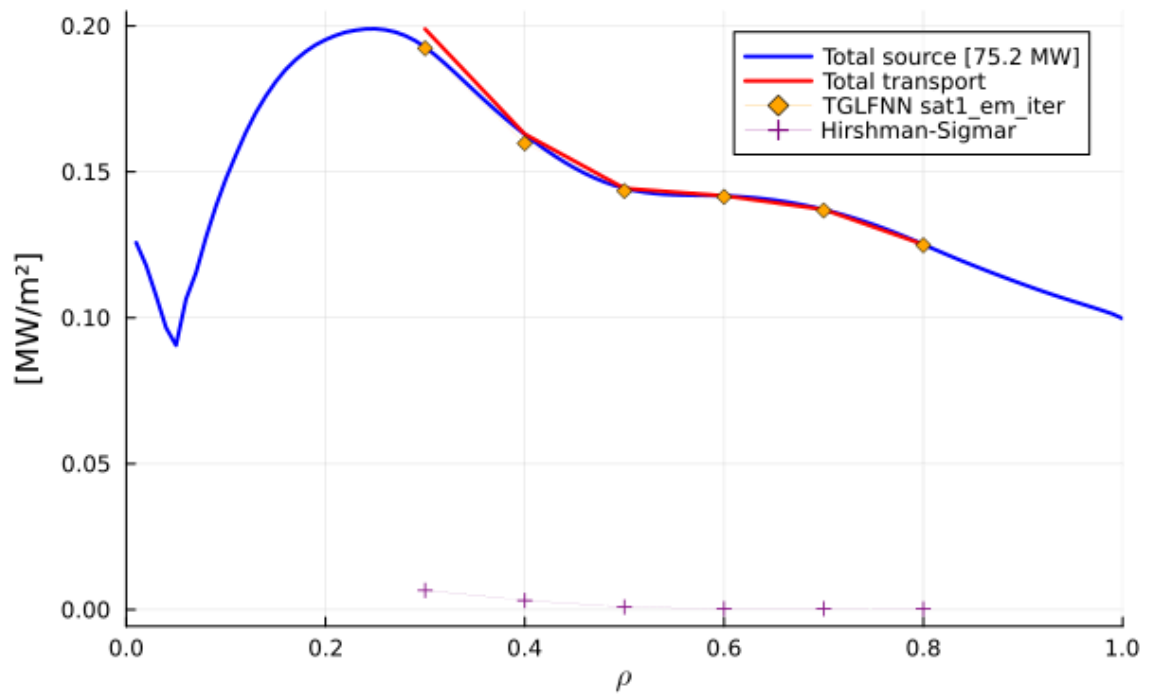
He4_fast Particles



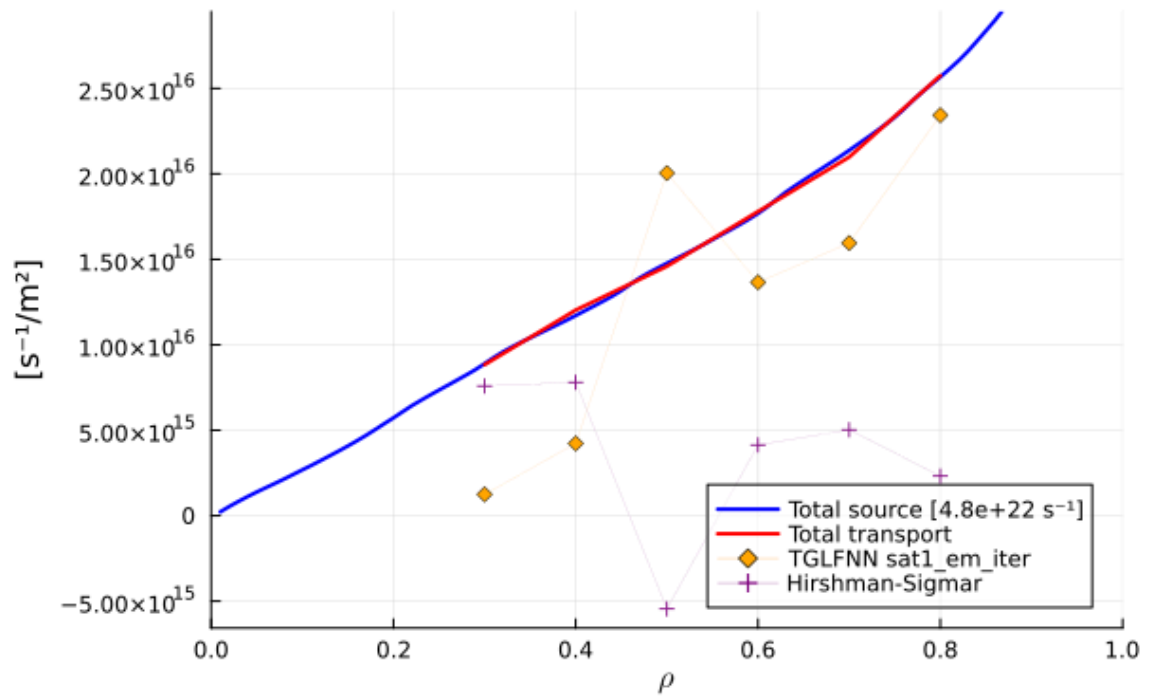
Electron energy flux



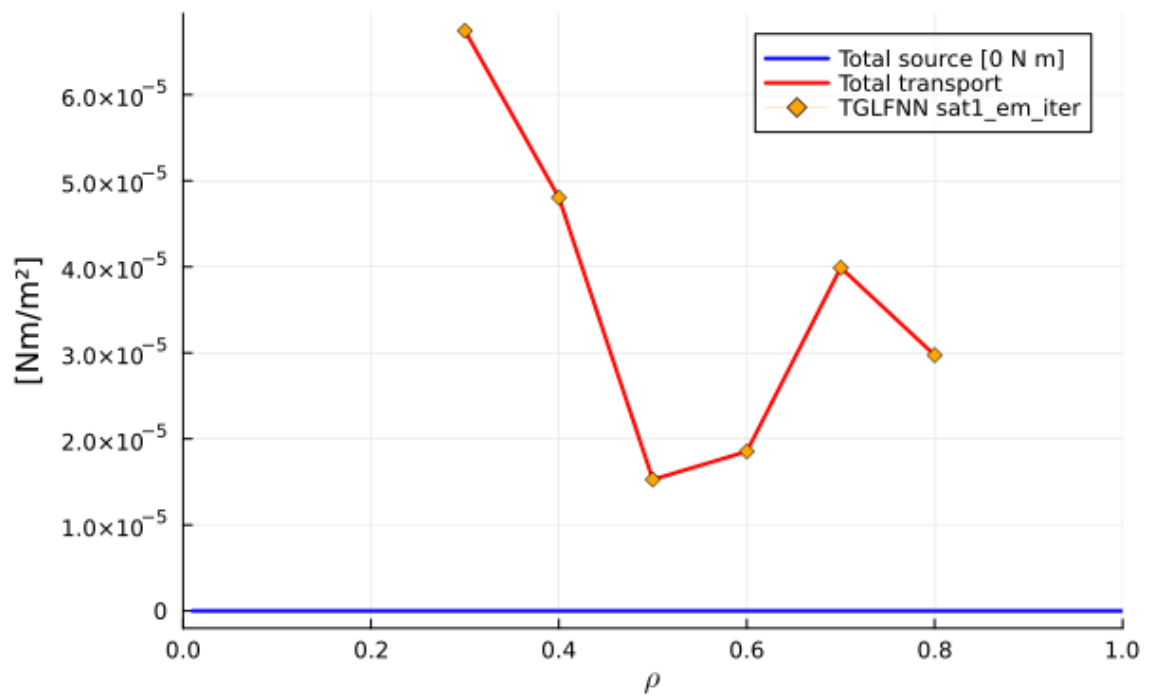
Total ion energy flux

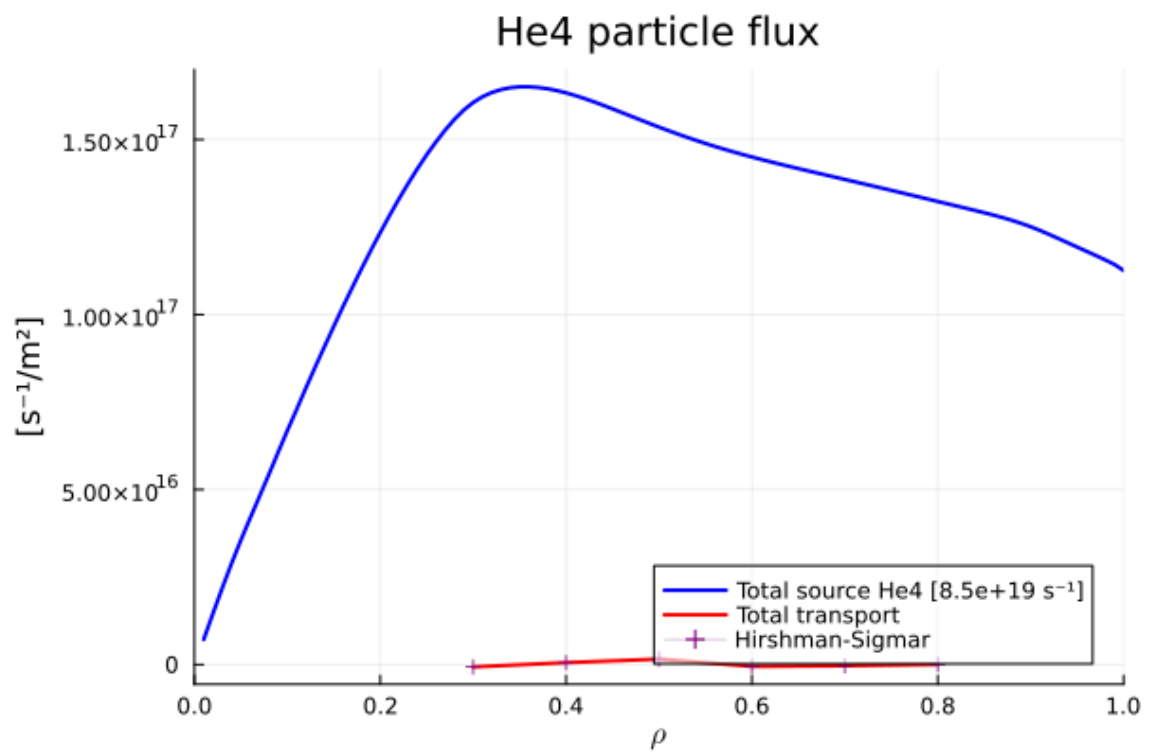
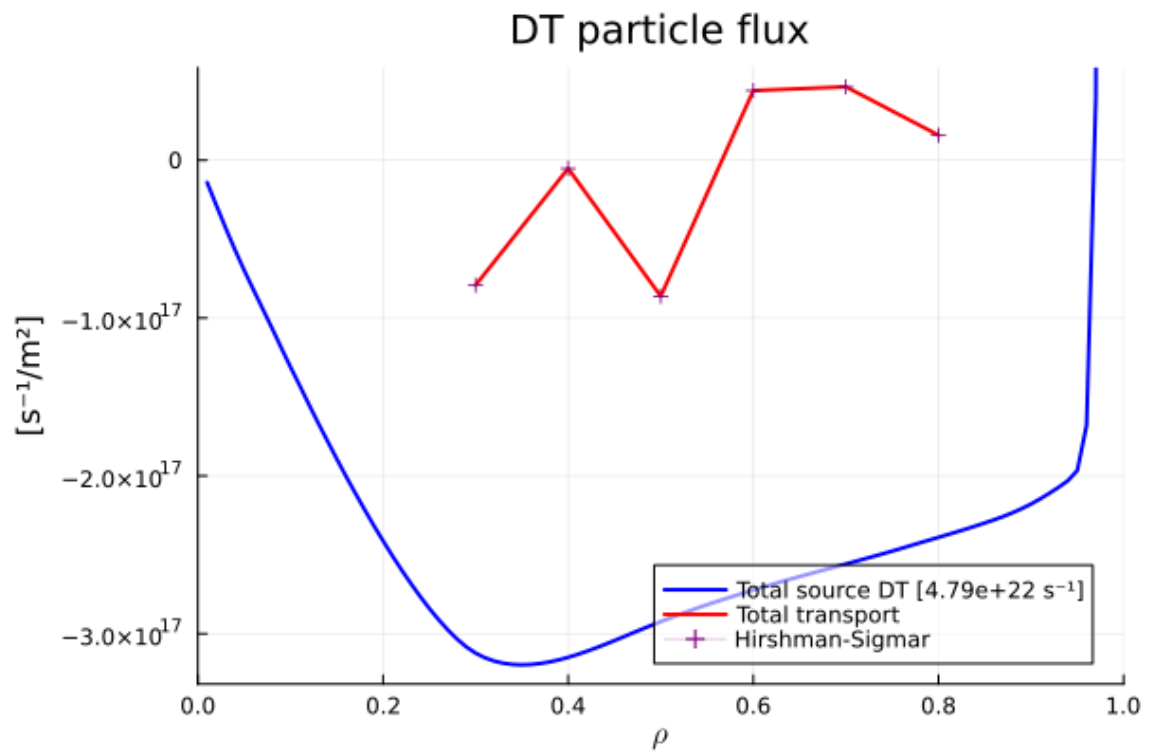


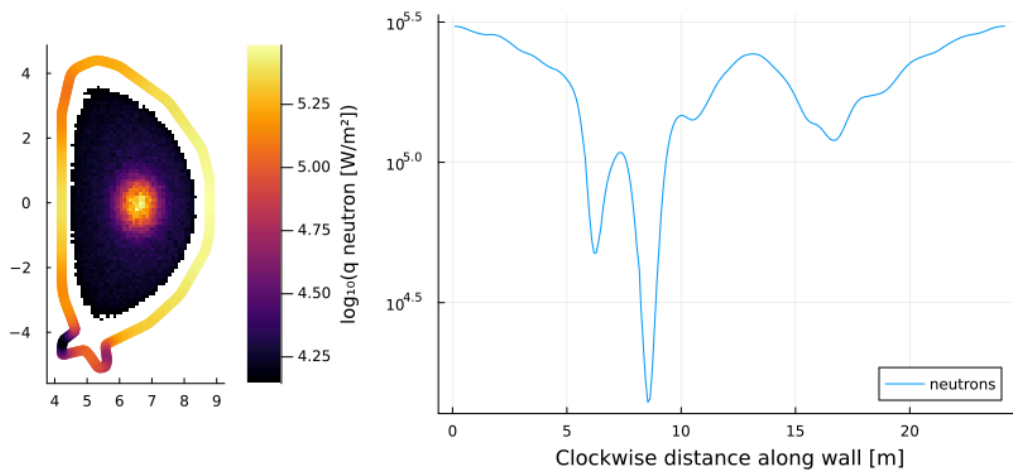
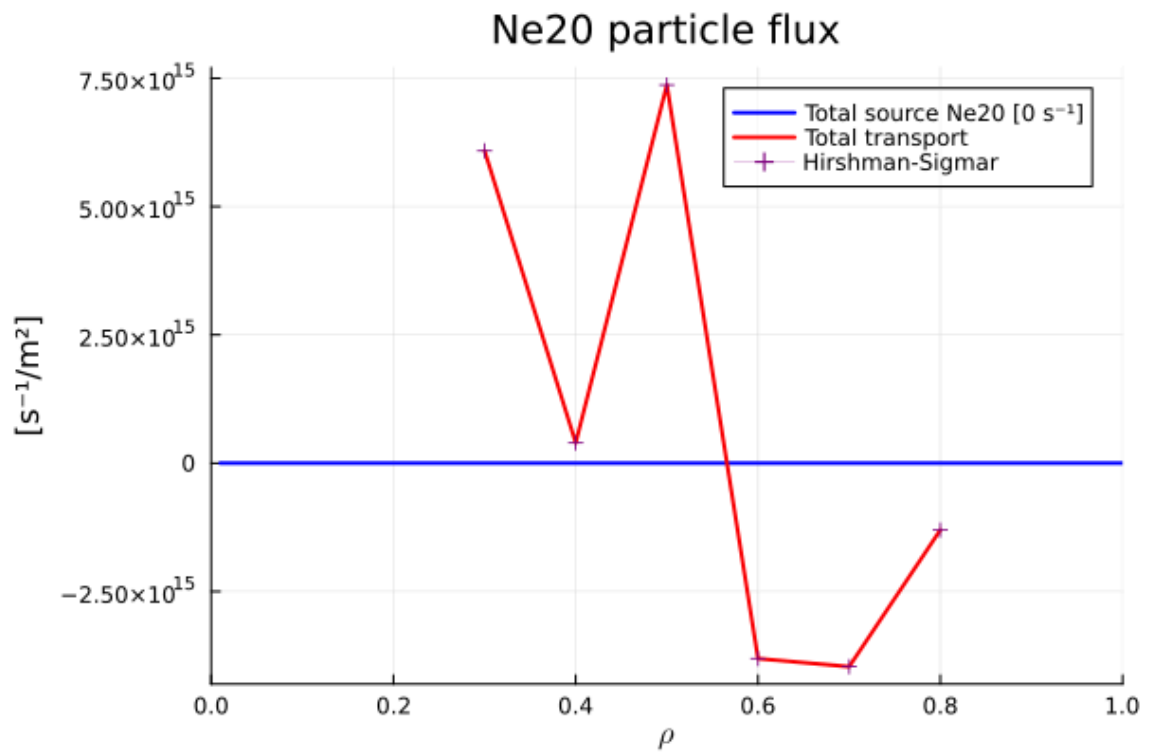
Electron particle flux

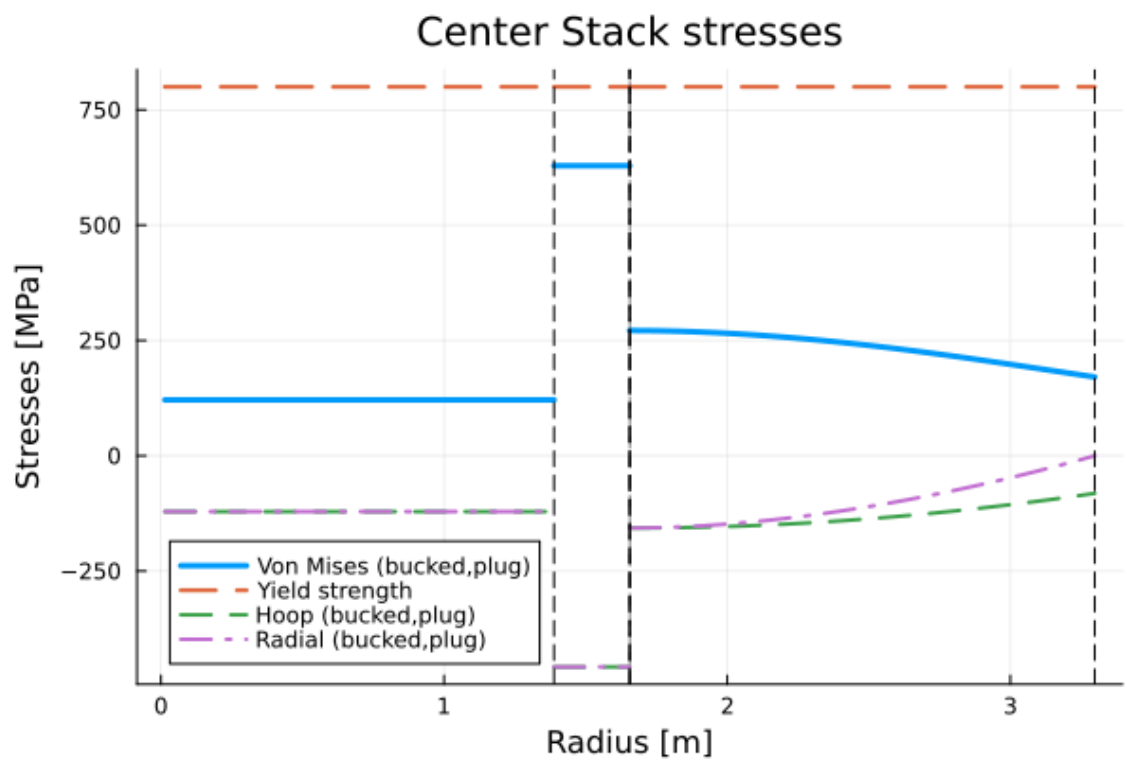
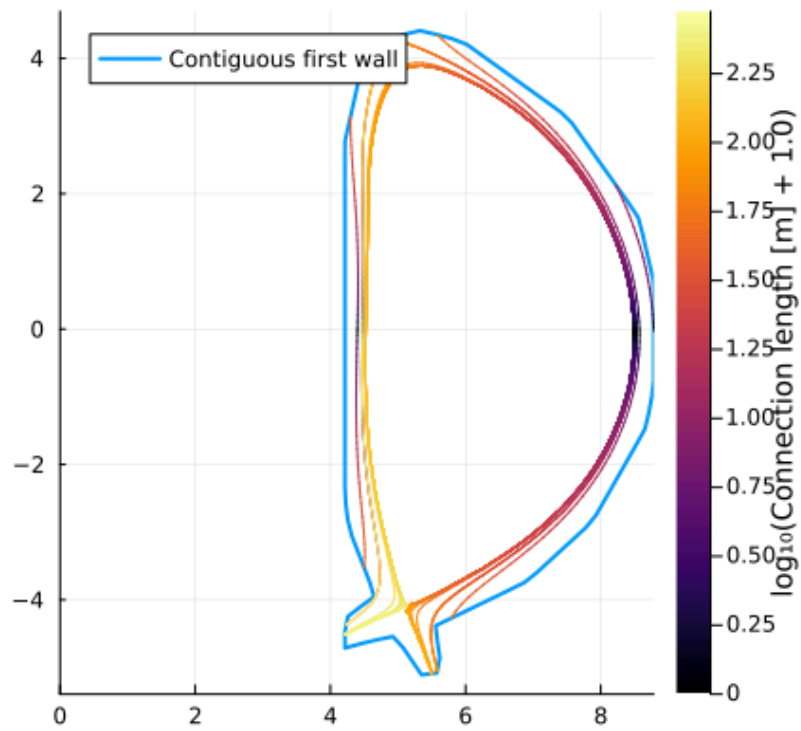


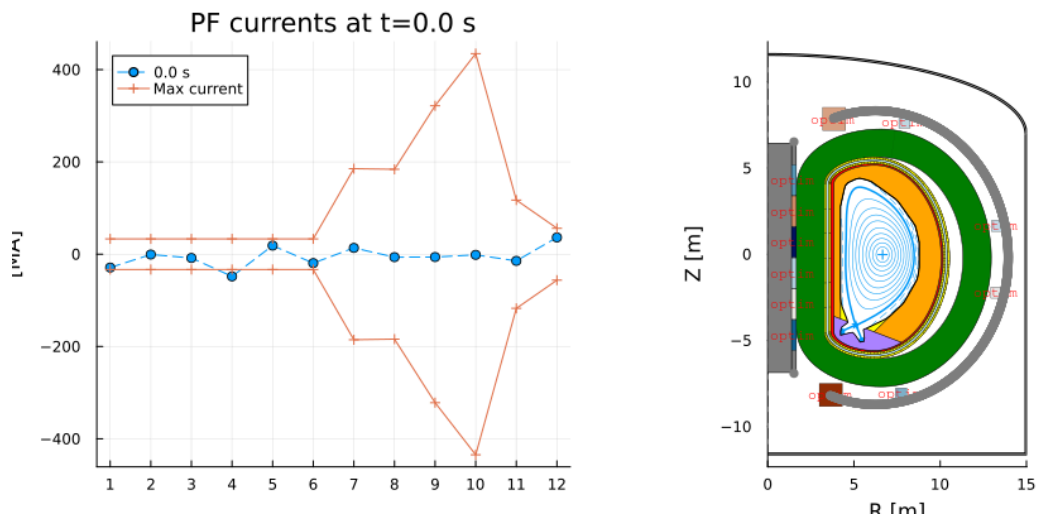
Momentum flux





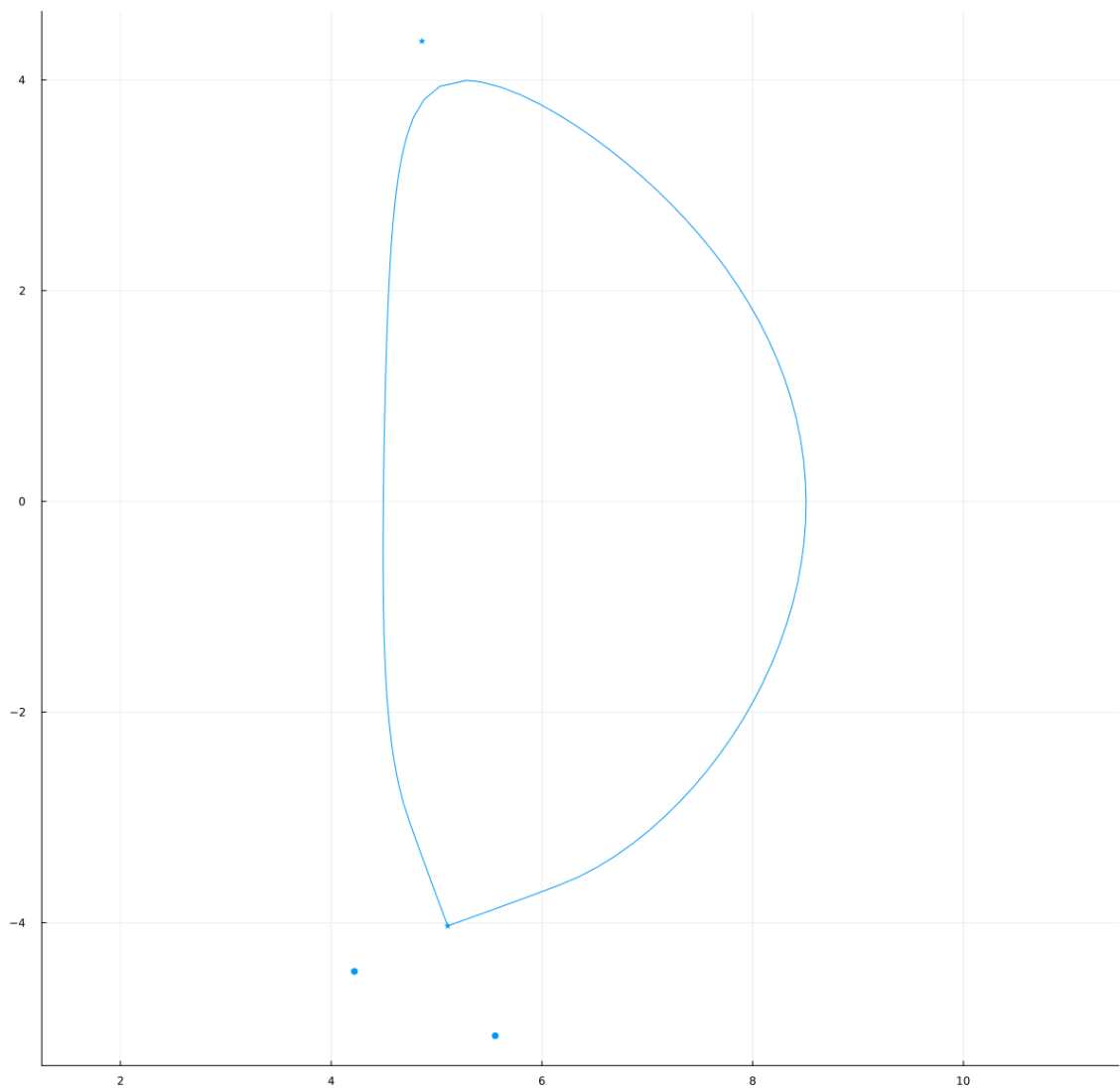


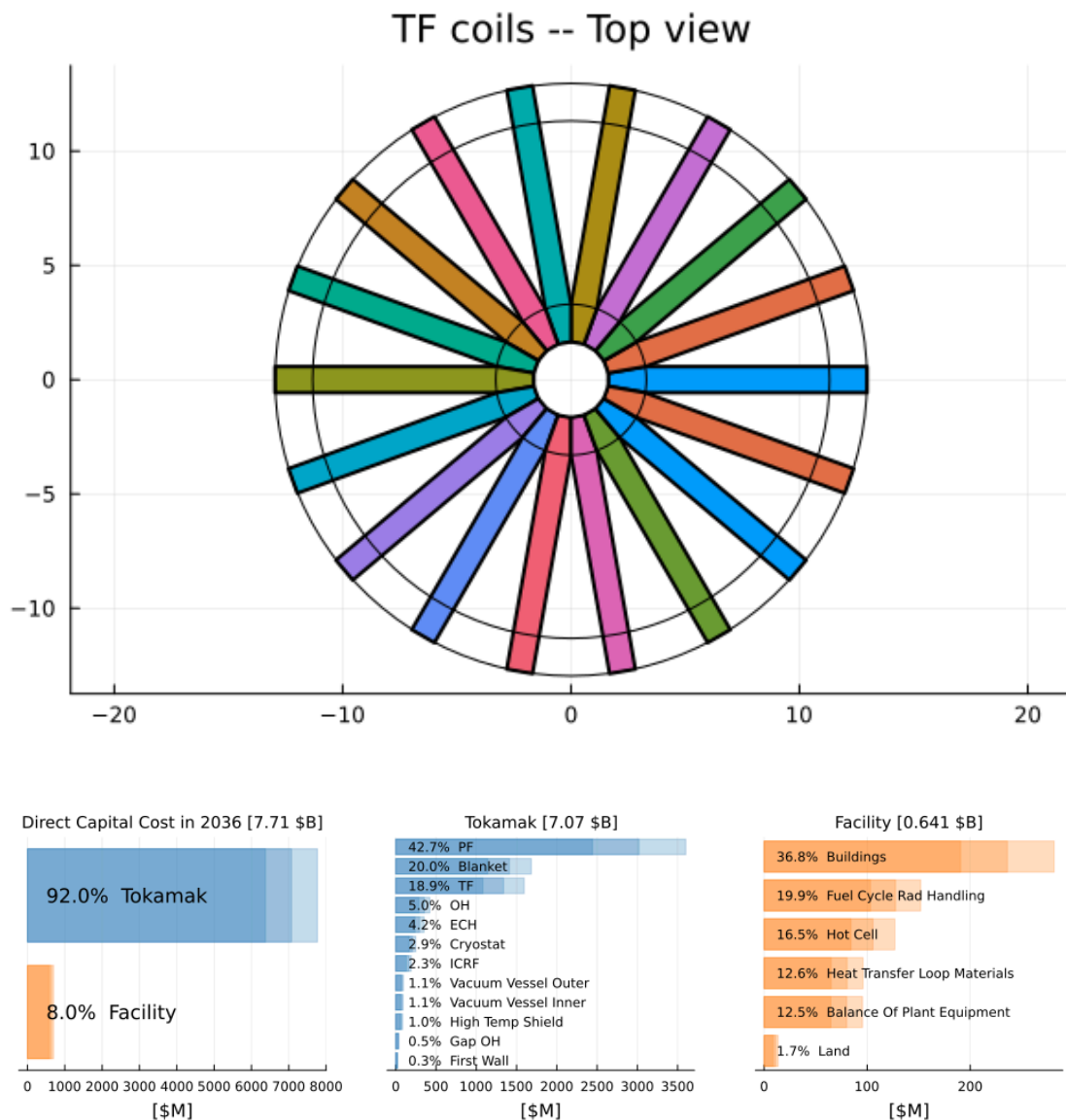




GKS: could not find font middle.ttf

Pulse schedule





Like before we can checkpoint results for later use

```
In [36]: @checkin :awf dd ini act
```

Working with the IMAS data structure

See the IMAS tutorial

https://github.com/ProjectTorreyPines/FuseExamples/blob/master/tutorial_imas.ipynb

Understanding how to work with the IMAS data structure is a must for working within the FUSE ecosystem!

Summary

Snapshot of `dd` in OD quantities (evaluated at `dd.global_time`).

Extract + plots saved to PDF (printed to screen if `filename` is omitted). NOTE: For PDF creation to work, one may need to install of `DejaVu Sans Mono` font.

```
In [37]: tutorial_temp_dir = tempdir()
filename = joinpath(tutorial_temp_dir, "${ini.general.casename}.pdf")
display(filename)
FUSE.digest(dd)#, filename)
```

"C:\\Users\\Croma\\AppData\\Local\\Temp\\K-DEMO.pdf"

GEOMETRY EMPERATURES

R0 → 6.5 [m]
e0 → 17.5 [keV]
a → 2.01 [m]
i0 → 17.9 [keV]
1/ε → 3.24
Te> → 8.19 [keV]
κ → 2
Ti> → 7.5 [keV]
δ → 0.588
e0/<Te> → 2.13
ζ → -0.0136
i0/<Ti> → 2.38
Volume → 924 [m³]
Surface → 753 [m²]

DENSITIES RANSPORT

ne0 → 9.1e+19 [m⁻³]
e → 2.33 [s]
ne_ped → 6.68e+19 [m⁻³]
e_exp → 1.81 [s]
ne_line → 8.28e+19 [m⁻³]
98y2 → 0.83
<ne> → 7.57e+19 [m⁻³]
98y2_exp → 0.767
ne0/<ne> → 1.2
ds03 → 0.619
fGW → 0.827
ds03_exp → 0.552
zeff_ped → 2
α_thermalization → 0.84 [s]
<zeff> → 2
α_slowing_down → 0.901 [s]
impurities → DT Ne20 He4

SOURCES URRENTS

Pec → 50 [MW]
p_bs_aux_ohm → 12.9 [MA]
rho0_ec → 0.57 [MW]
p_ni → 7 [MA]
Pnbi → NaN [MW]
p_bs → 3.18 [MA]
Enbi1 → NaN [MeV]
p_aux → 3.82 [MA]
Pic → 50 [MW]
p_ohm → 5.95 [MA]
Plh → NaN [MW]
jima → 0.4
Paux_tot → 100 [MW]
lattop → 0.7 [Hours]
Pa → 47.9 [MW]

EQUILIBRIUM

B0 → 7.8 [T]
ip → 12.7 [MA]
q95 → 6.13
<Bpol> → 0.798 [T]
βpol_MHD → 0.757
βtor_MHD → 0.00834
βn_MHD → 1.02

PRESSURES

P0 → 0.554 [MPa]
<P> → 0.202 [MPa]
P0/<P> → 2.75
βn → 1.03
βn_th → 0.991

EXHAUST

Psol → 115 [MW]
PLH → 134 [MW]
Bpol_omp → 1.11 [T]
λq → 0.974 [mm]
qp01 → 2.22e+03 [MW/m²]
qpar → 1.21e+04 [MW/m²]
P/R0 → 17.7 [MW/m]
PB/R0 → 138 [MW T/m]

T

T

T

<

<

T

T

T

τ

τ

H

H

H

H

τ

τ

C

i

i

i

i

i

e

f

Pohm → 0.512 [MW]
Pheat → 148 [MW]
Prad_tot → -33.3 [MW]

PBp/R0 → 14.1 [MW T/m]
PBe/R0q95 → 6.95 [MW T/m]
neutrons_peak → 0.306 [MW/m²]

BOP
OSTING

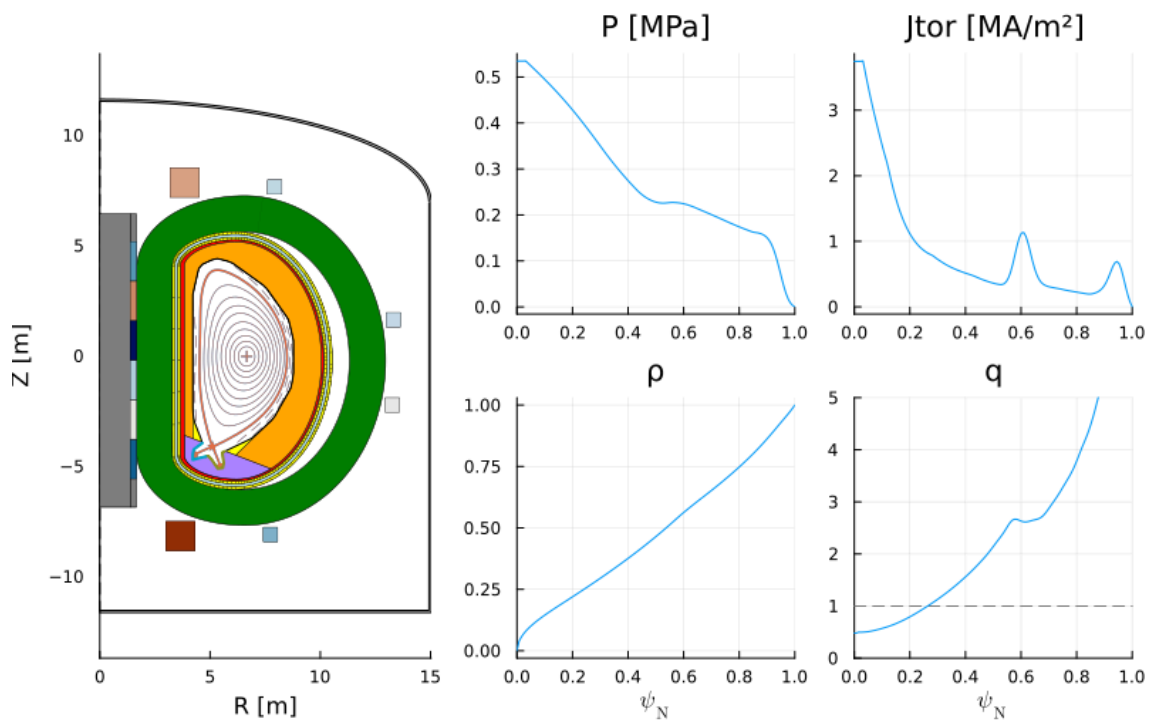
BUILD

C

Pfusion → 239 [MW]
apital_cost → 7.71 [\$B]
Qfusion → 2.39
evelized_CoE → Inf [\$/kWh]
thermal_cycle_type → rankine
F_of_total → 17.4 [%]
thermal_efficiency_plant → 19.4 [%]
OP_of_total → 1.04 [%]
thermal_efficiency_cycle → NaN [%]
lanket_of_total → 18.3 [%]
power_electric_generated → 8.32 [MW]
ryostat_of_total → 2.69 [%]
Pelectric_net → -137 [MW]
Qplant → 0.0574
TBR → 0.0685

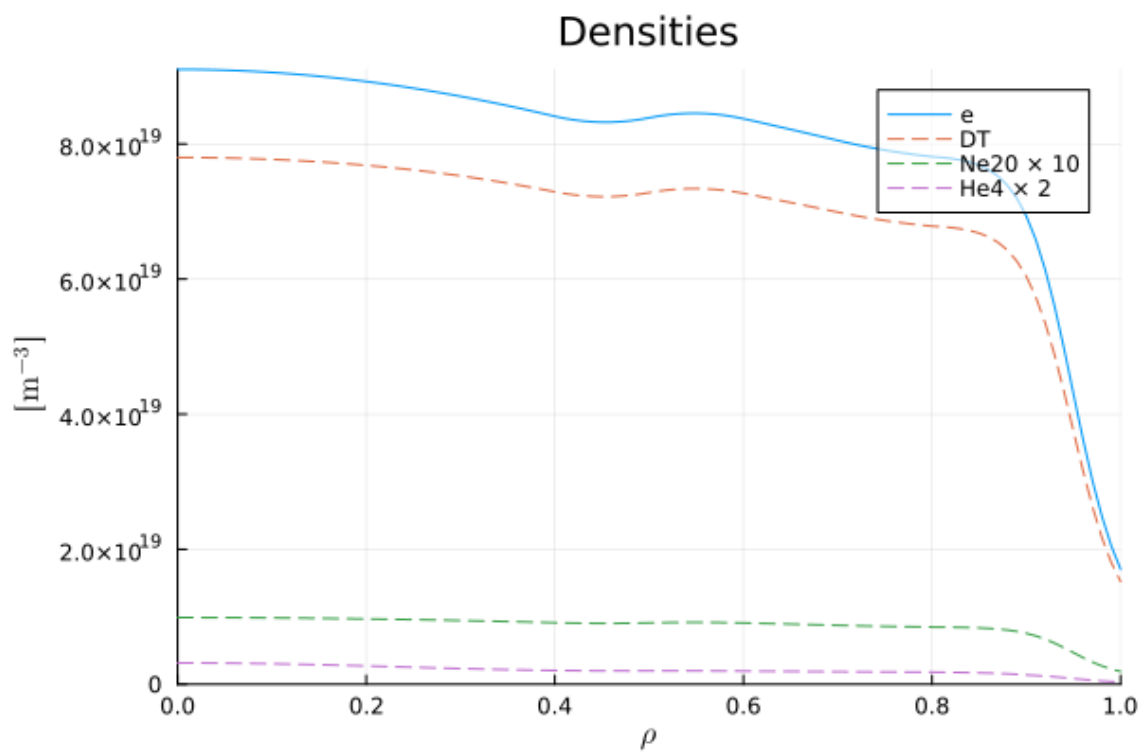
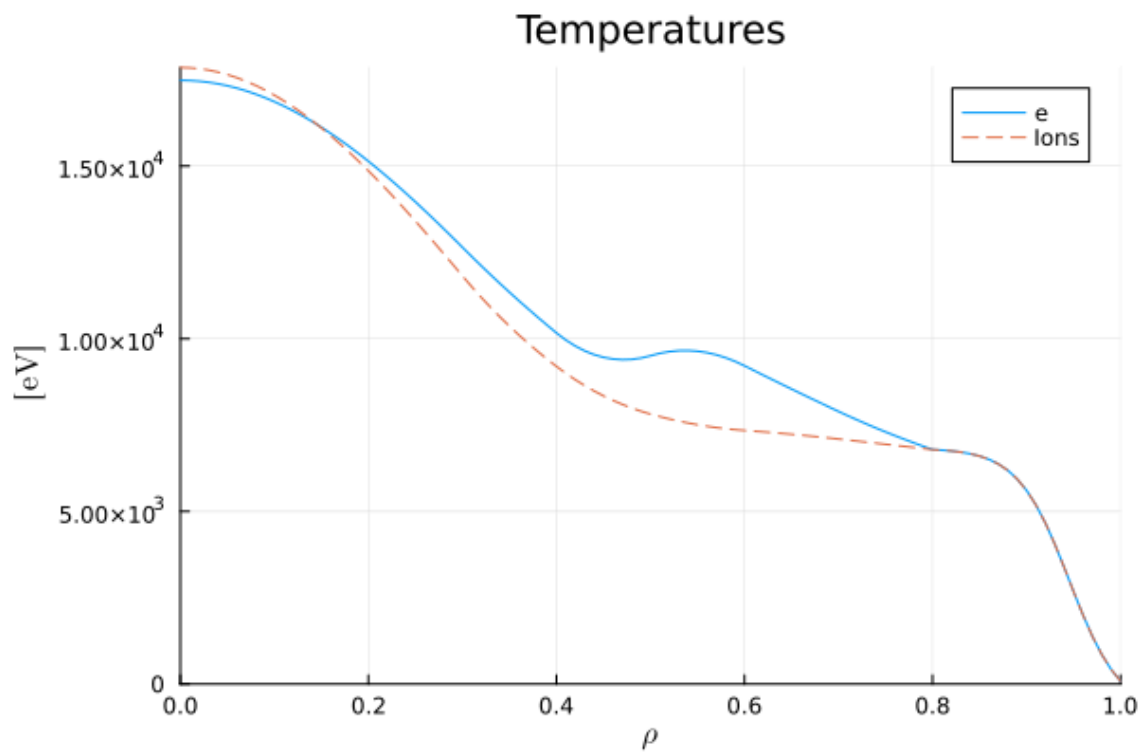
PF_material → nb3sn **c**
TF_material → nb3sn_kdemo **l**
OH_material → nb3sn **T**
TF_max_b → 15.4 [T] **B**
OH_max_b → 14.8 [T] **b**
TF_j_margin → 6.14 **c**
OH_j_margin → 1.4
TF_stress_margin → 2.95
OH_stress_margin → 1.27

@ time = 0.0 [s]

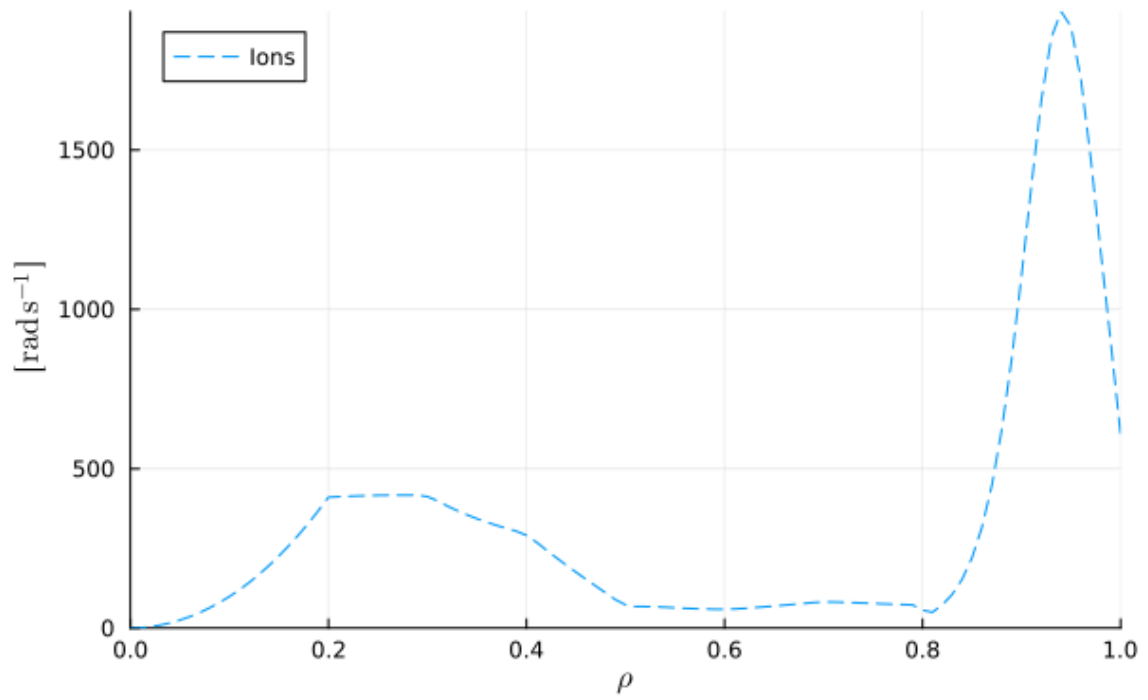


23x10 DataFrame

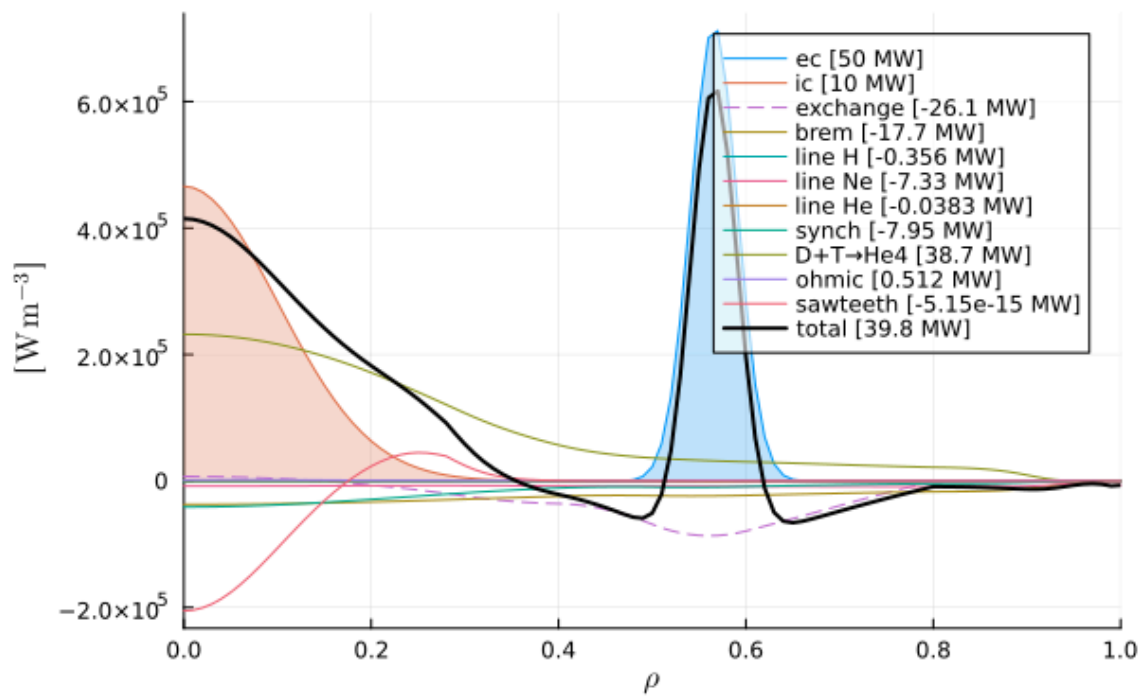
Row	group	details			type	ΔR	R_start
R_end	material	area	volume	shape	String	Float64	Float64
Float64	String	Float64	Float64	String			
1	in					1.38838	0.0
1.38838	steel	18.4649	80.5389	rectangle			
2	in			oh		0.267154	1.38838
1.65554	nb3sn	3.55303	33.9767	rectangle			
3	hfs			tf		1.64241	1.65554
3.29794	nb3sn_kdemo	62.4071	1293.17	convex hull			
4	hfs	gap tf	vacuum vessel			0.0	3.29794
3.29794	vacuum	7.52314	497.067	double ellipse			
5	hfs	vacuum	outer	vessel		0.0982993	3.29794
3.39624	steel	3.0898	126.315	negative offset			
6	hfs	gap water				0.147449	3.39624
3.54369	water	4.52083	184.937	negative offset			
7	hfs	vacuum	inner	vessel		0.0982993	3.54369
3.64199	steel	2.93797	120.268	negative offset			
8	hfs	gap high temp shield	vacuum vess...			0.00982993	3.64199
3.65182	vacuum	0.607407	31.2805	negative offset			
9	hfs	high temp		shield		0.196599	3.65182
3.84842	steel	3.84481	140.127	negative offset			
10	hfs			blanket		0.373538	3.84842
4.22196	lithium_lead	23.2776	1071.7	negative offset			
11	hfs	first		wall		0.0196599	4.22196
4.24162	tungsten	0.980479	32.0663	offset			
12	lhfs			plasma		4.51651	4.24162
8.75812	plasma	32.2579	1256.89				
13	lfs	first		wall		0.0199664	8.75812
8.77809	tungsten	0.977276	31.9838	offset			
14	lfs			blanket		1.27329	8.77809
10.0514	lithium_lead	23.2808	1071.78	negative offset			
15	lfs	high temp		shield		0.102599	10.0514
10.154	steel	3.84481	140.127	negative offset			
16	lfs	gap high temp shield	vacuum vess...			0.0442026	10.154
10.1982	vacuum	0.607407	31.2805	negative offset			
17	lfs	vacuum	inner	vessel		0.0982993	10.1982
10.2965	steel	2.93797	120.268	negative offset			
18	lfs	gap water				0.147449	10.2965
10.4439	water	4.52083	184.937	negative offset			
19	lfs	vacuum	outer	vessel		0.0982993	10.4439
10.5422	steel	3.0898	126.315	negative offset			
20	lfs	gap tf	vacuum vessel			0.775582	10.5422
11.3178	vacuum	7.52314	497.067	double ellipse			
21	lfs			tf		1.64241	11.3178
12.9602	nb3sn_kdemo	62.4071	1293.17	convex hull			
22	out					1.96599	0.0
14.9262	vacuum	166.102	8803.51				
23	out			cryostat		0.0982993	0.0
15.0245	steel	5.00508	328.552	silo			



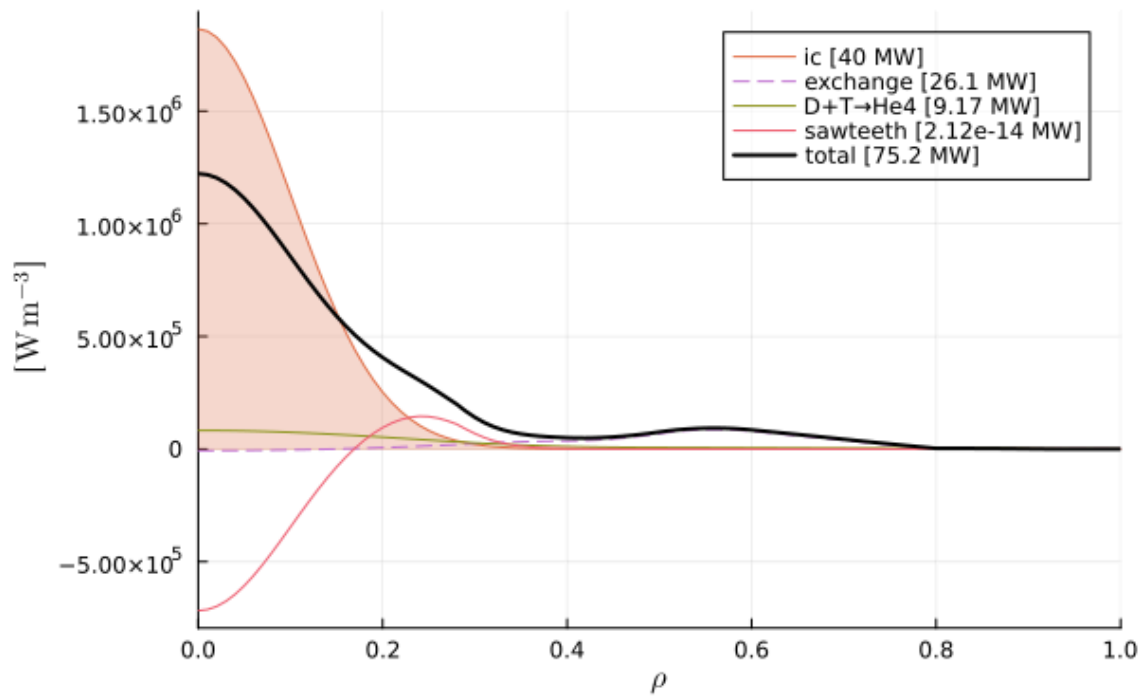
Toroidal rotation



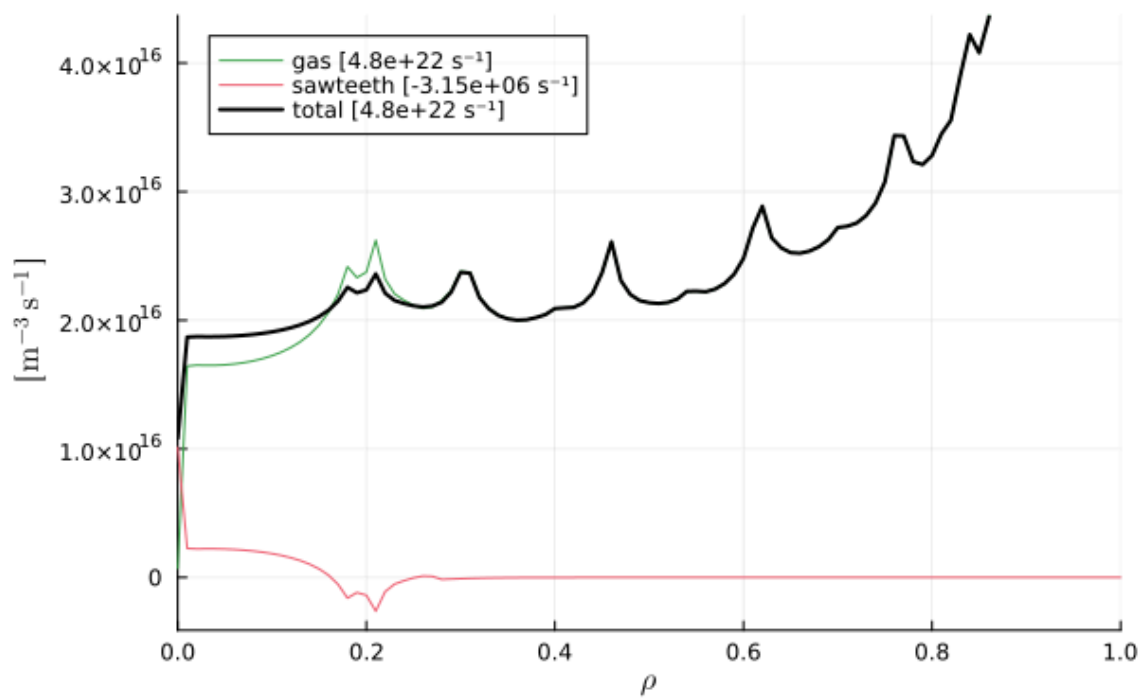
Electron Energy



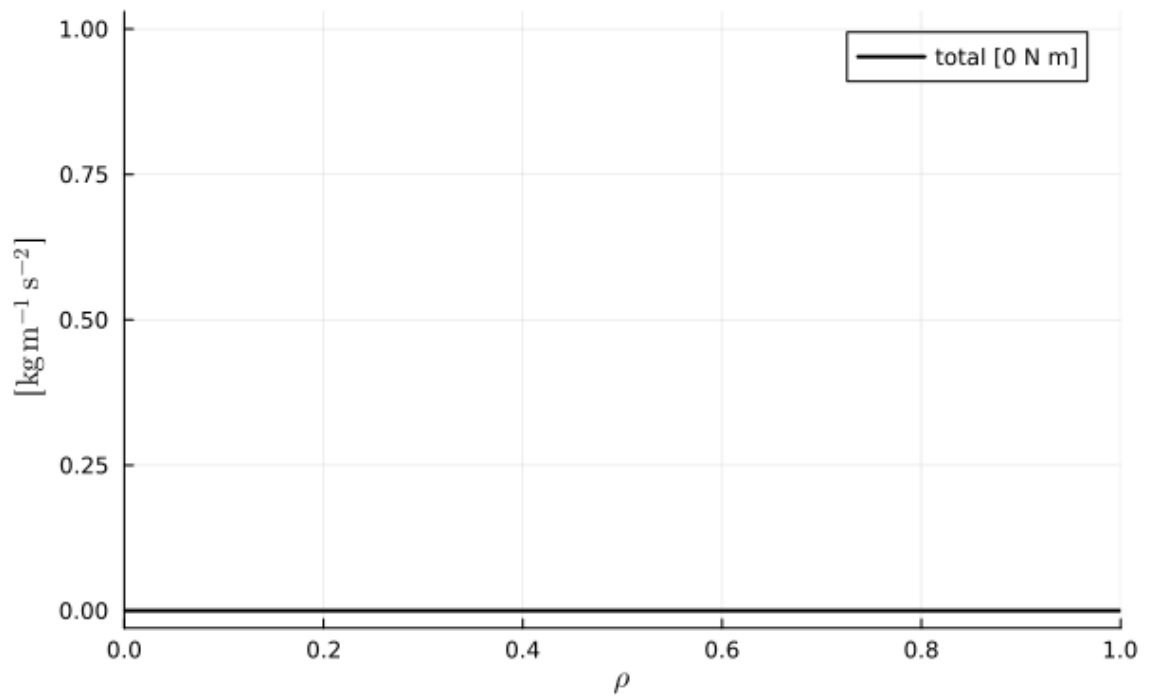
Total Ion Energy



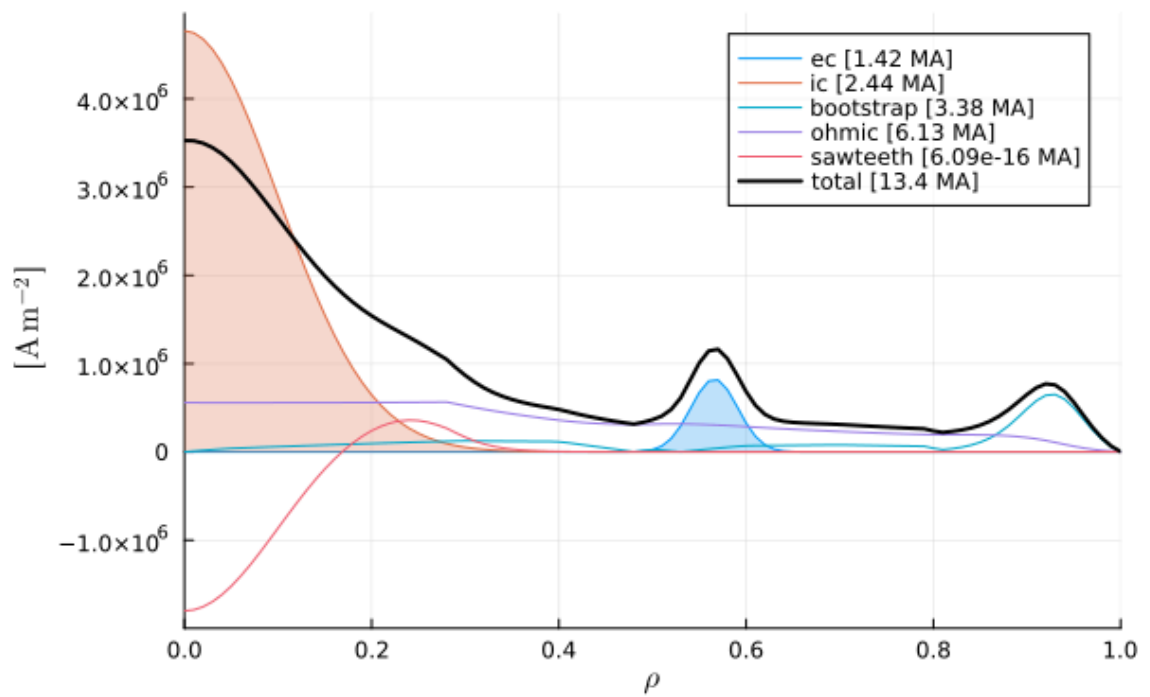
Electron Particles



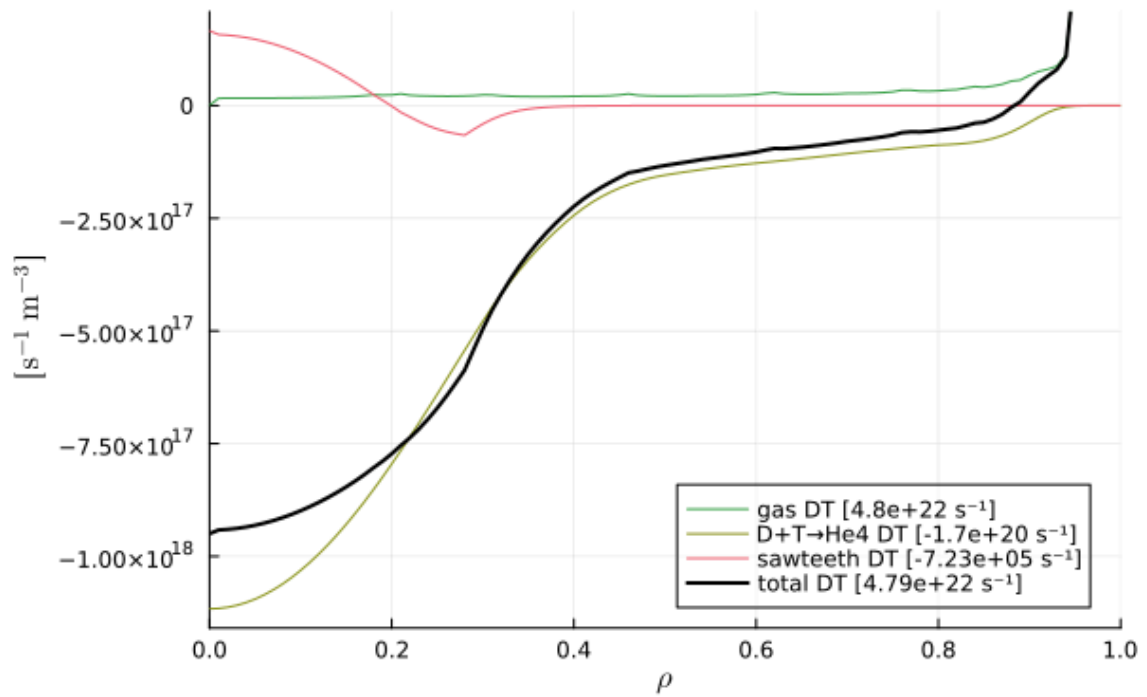
Momentum



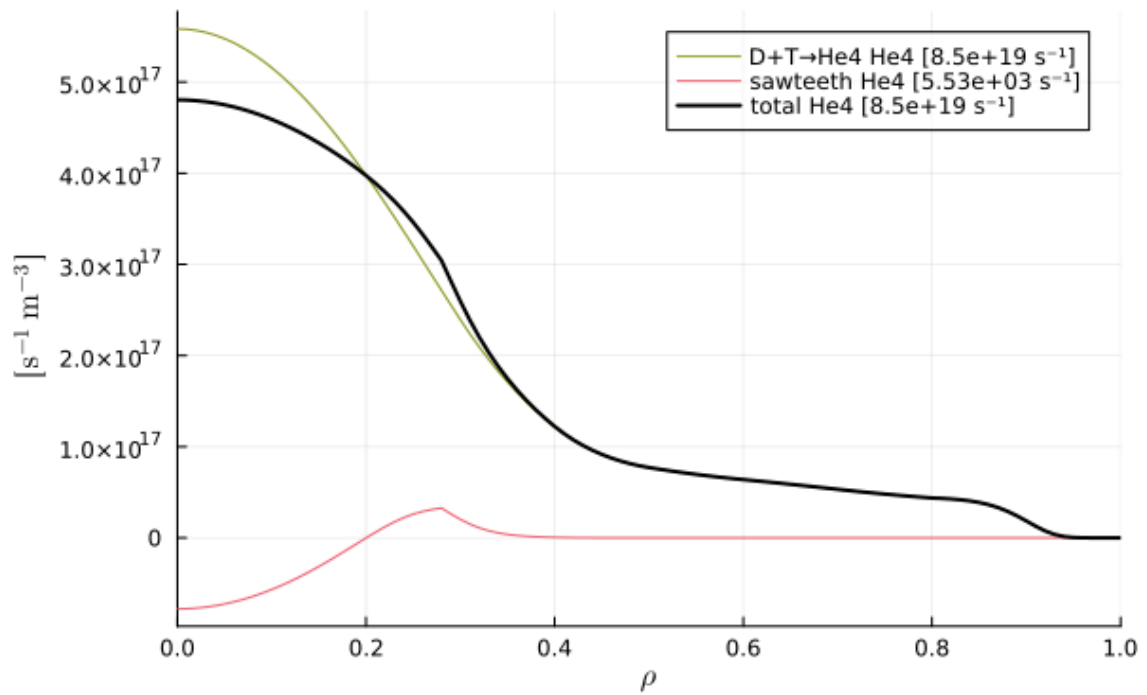
Parallel Current



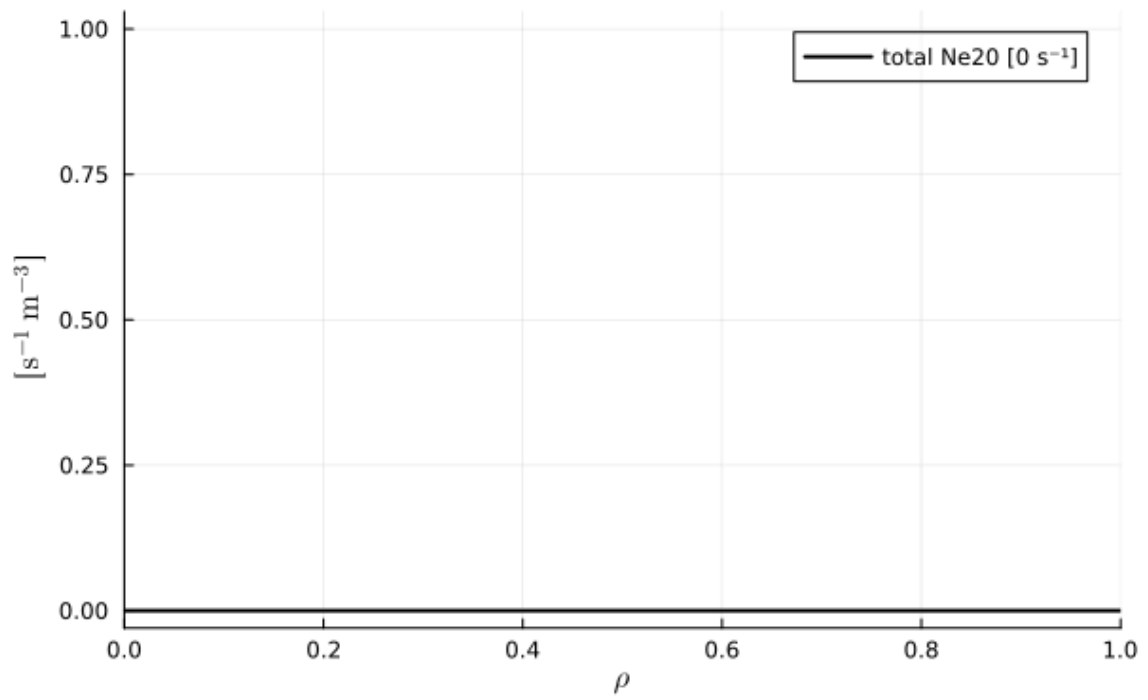
DT Particles



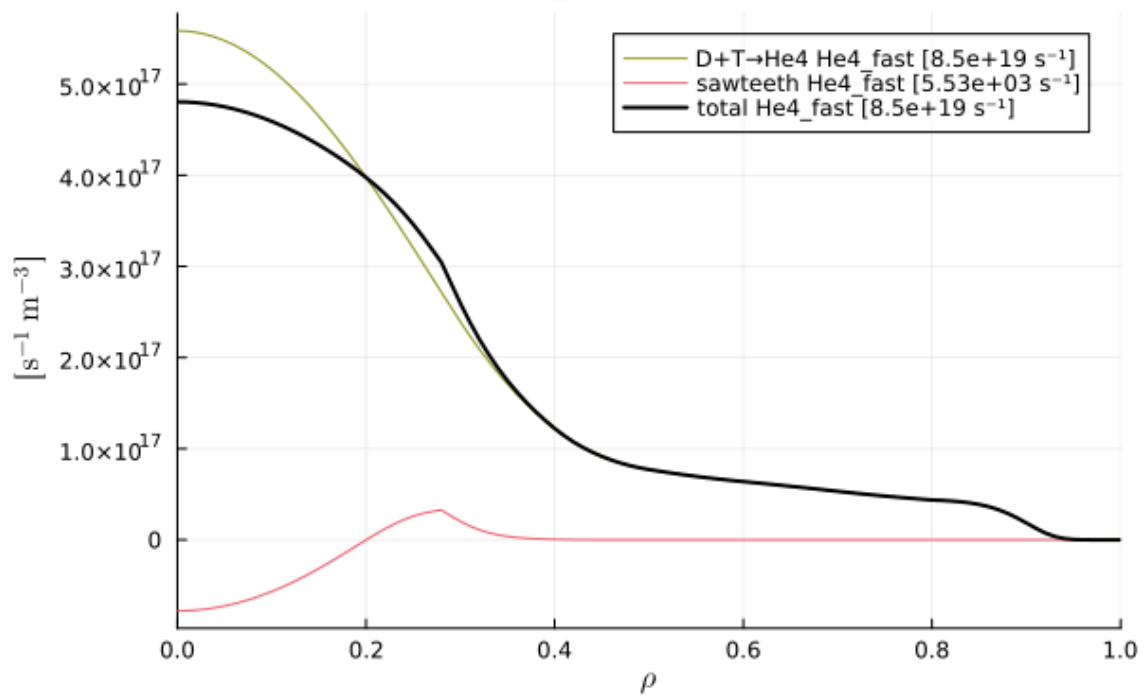
He4 Particles



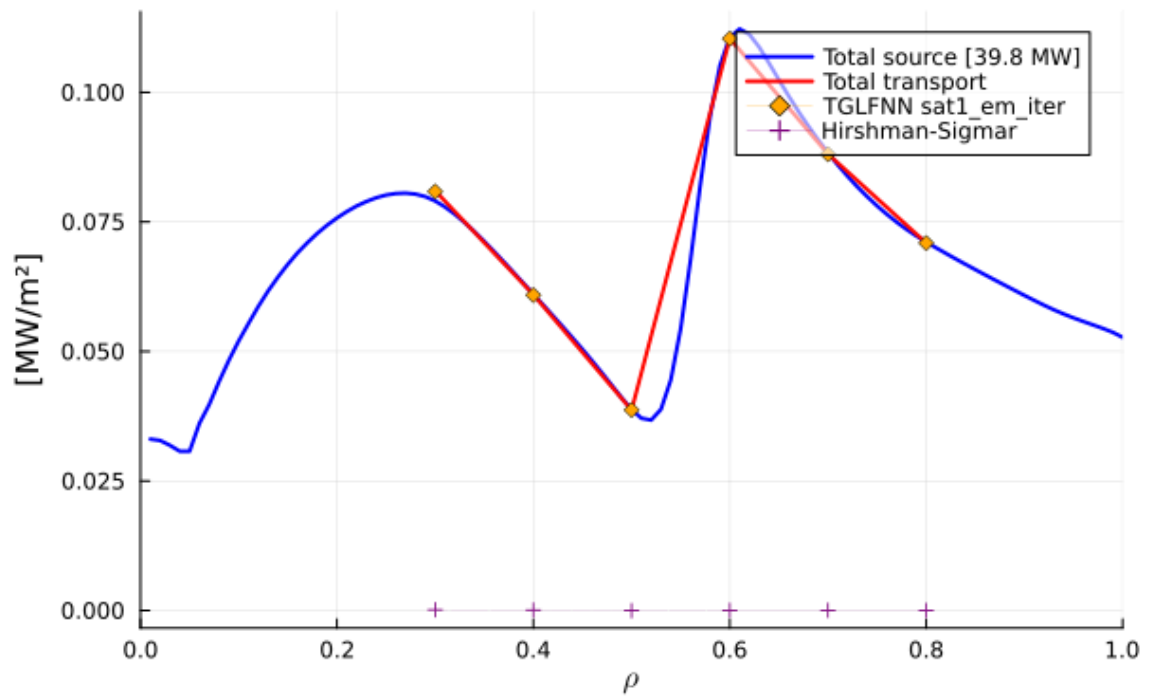
Ne20 Particles



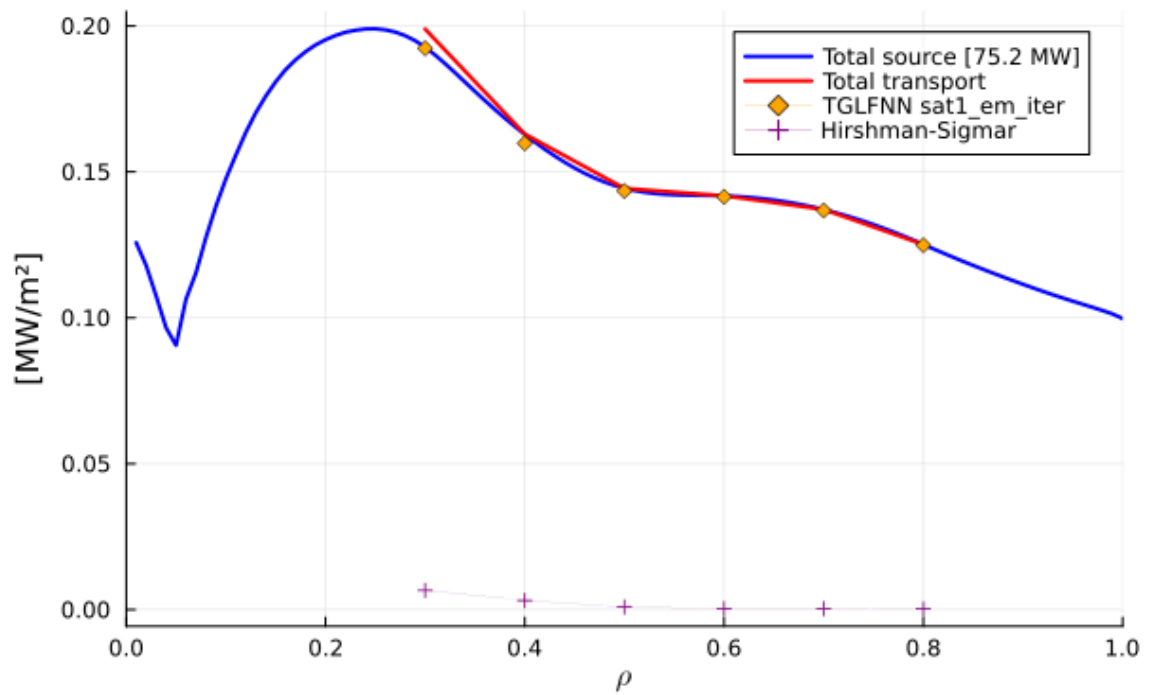
He4_fast Particles



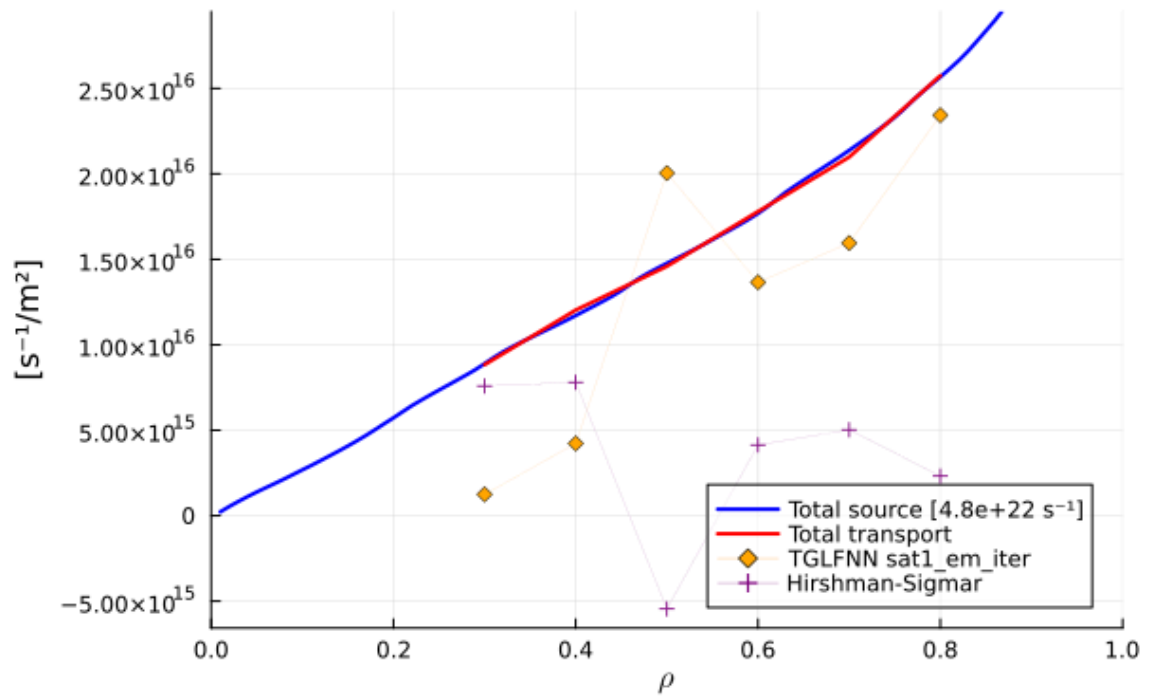
Electron energy flux



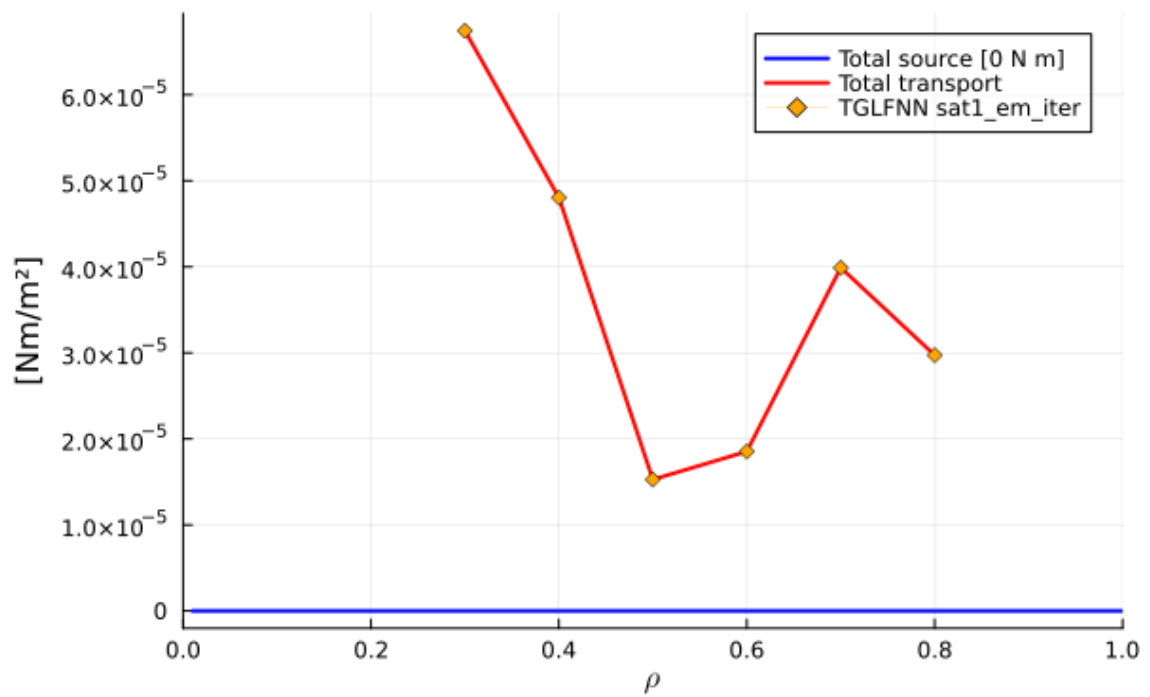
Total ion energy flux

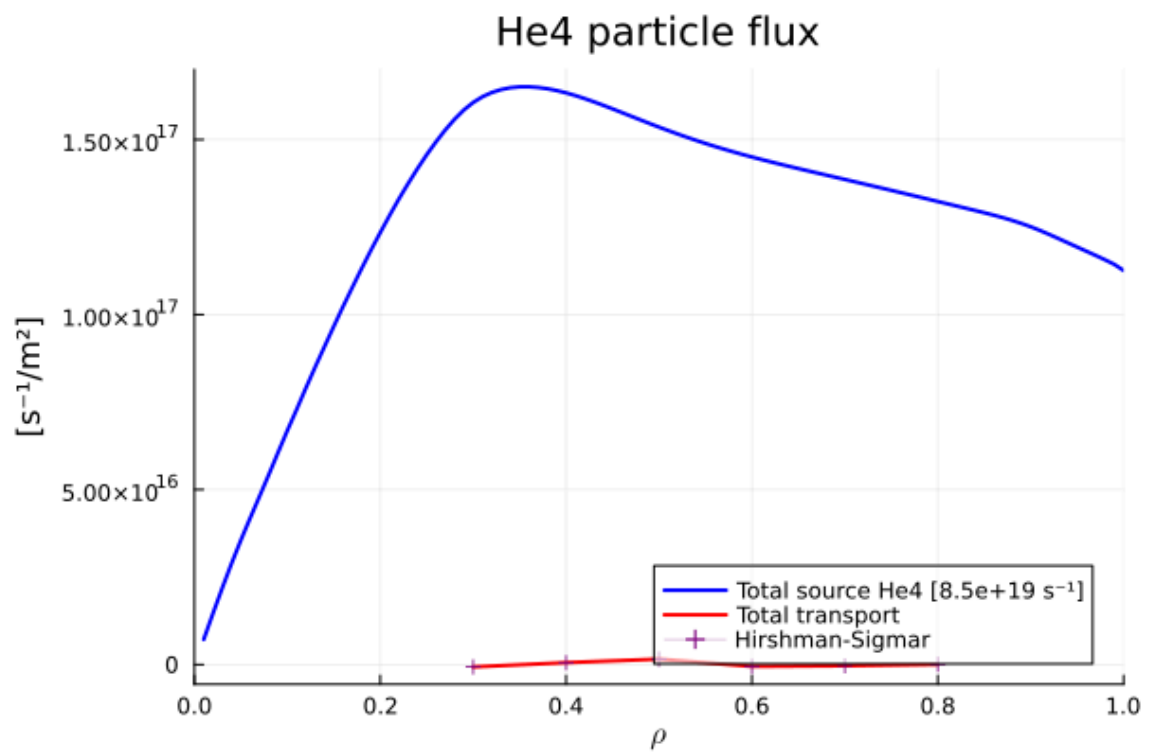
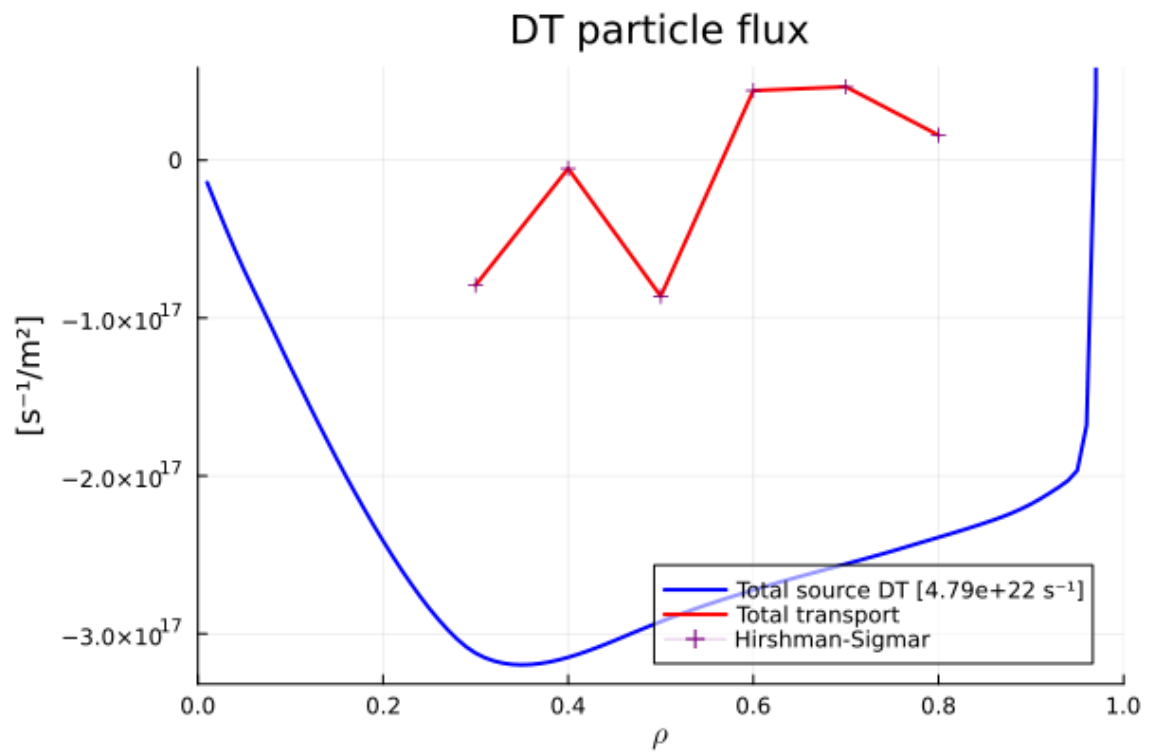


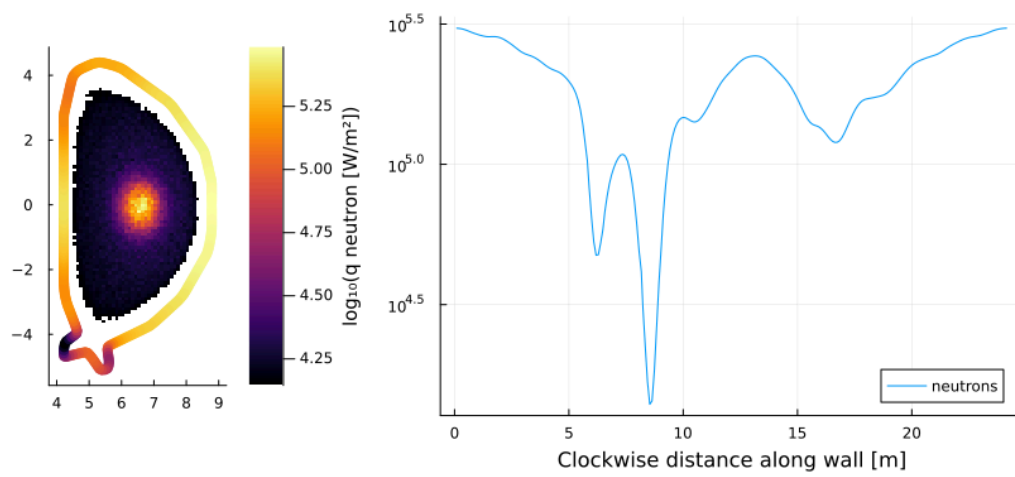
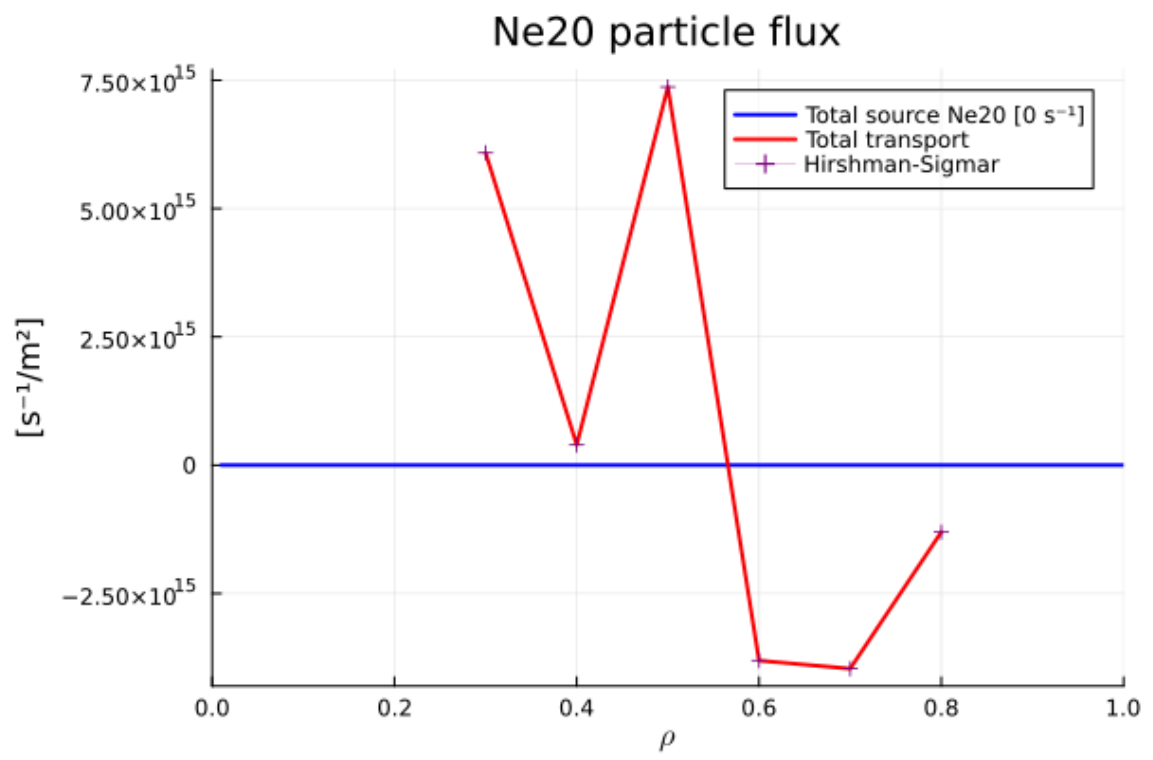
Electron particle flux

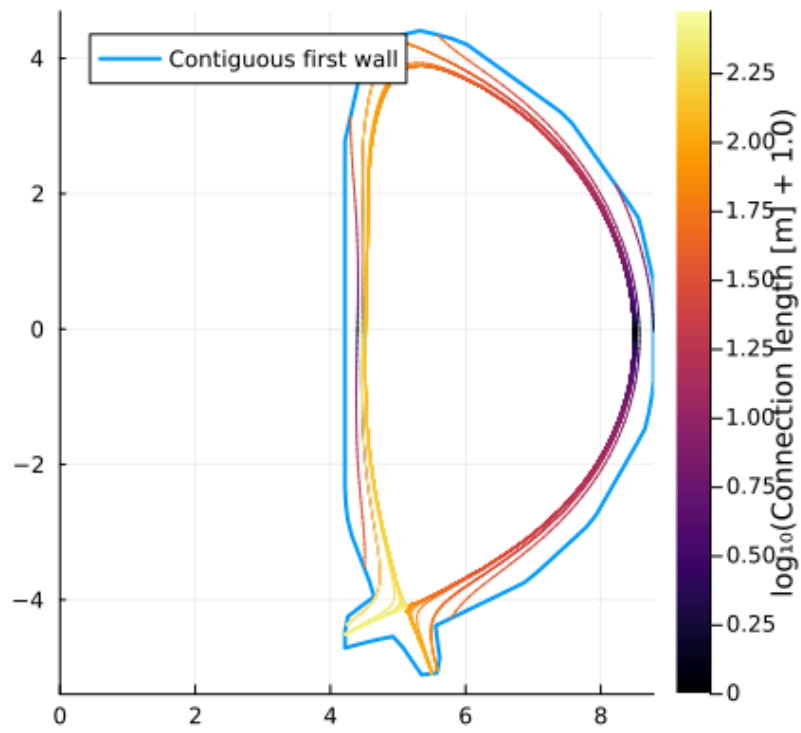


Momentum flux

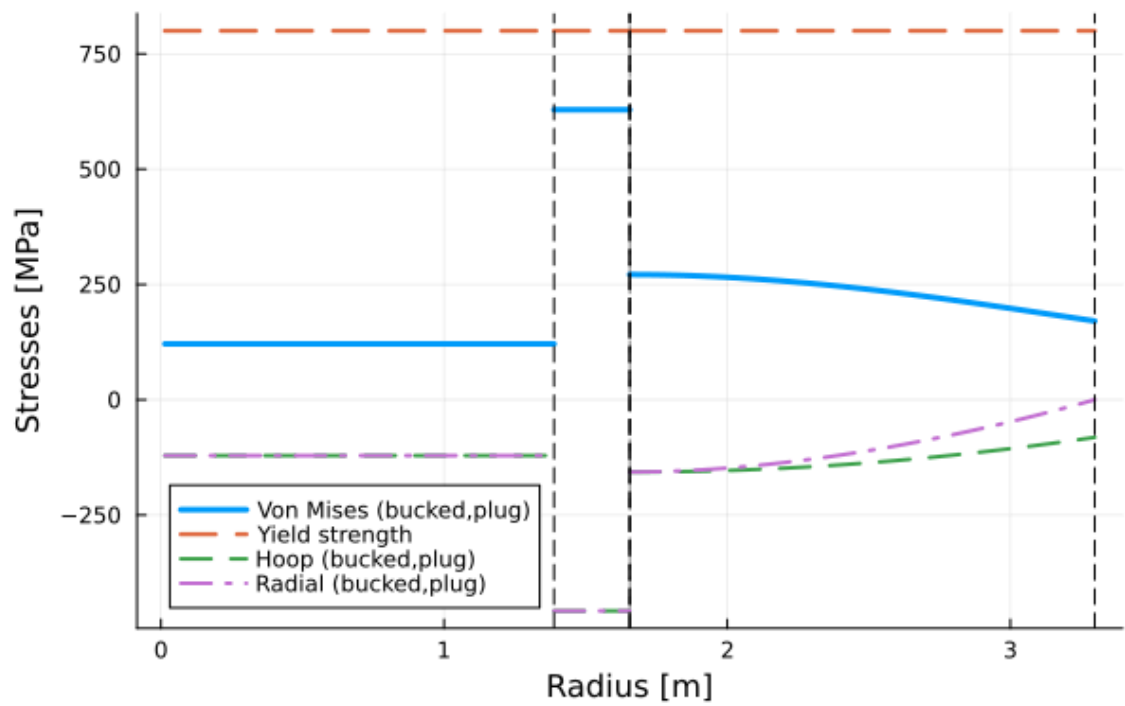


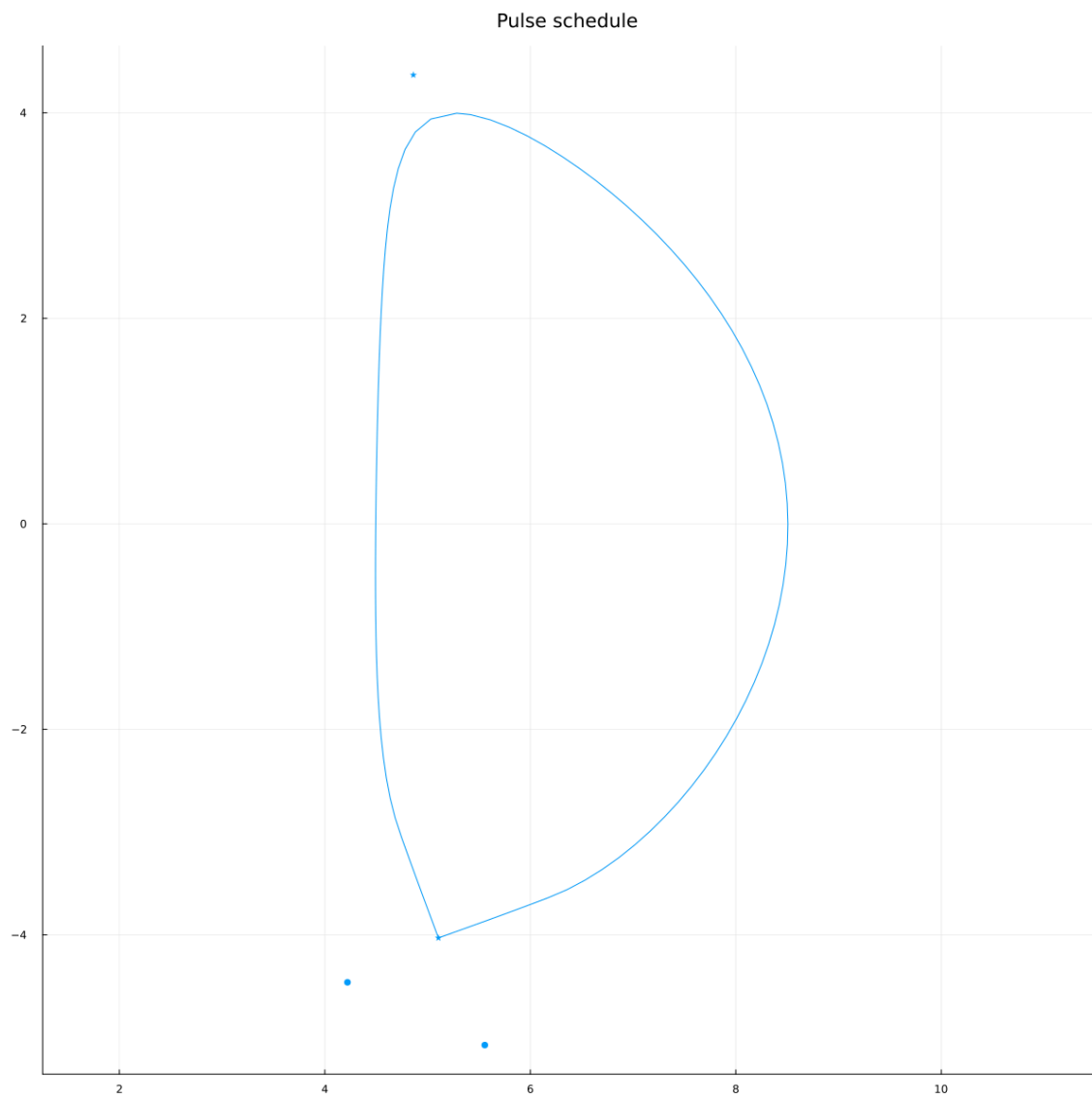
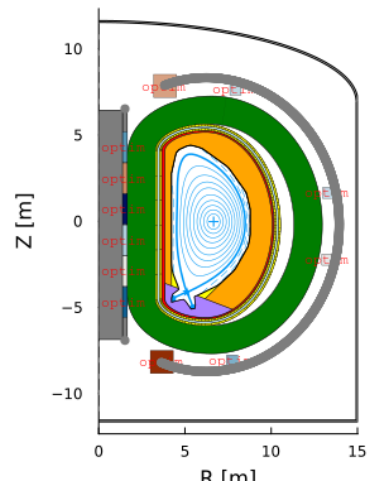
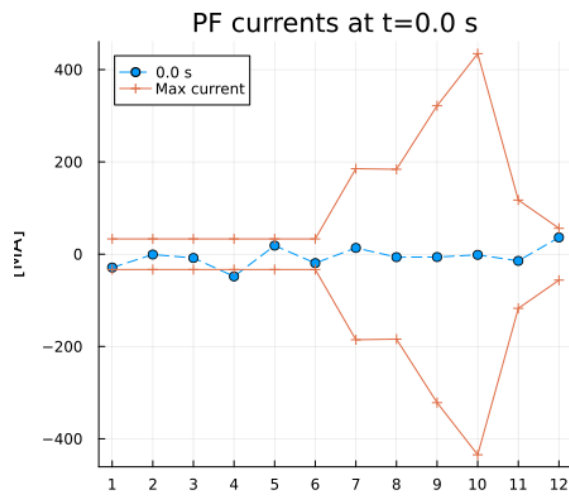




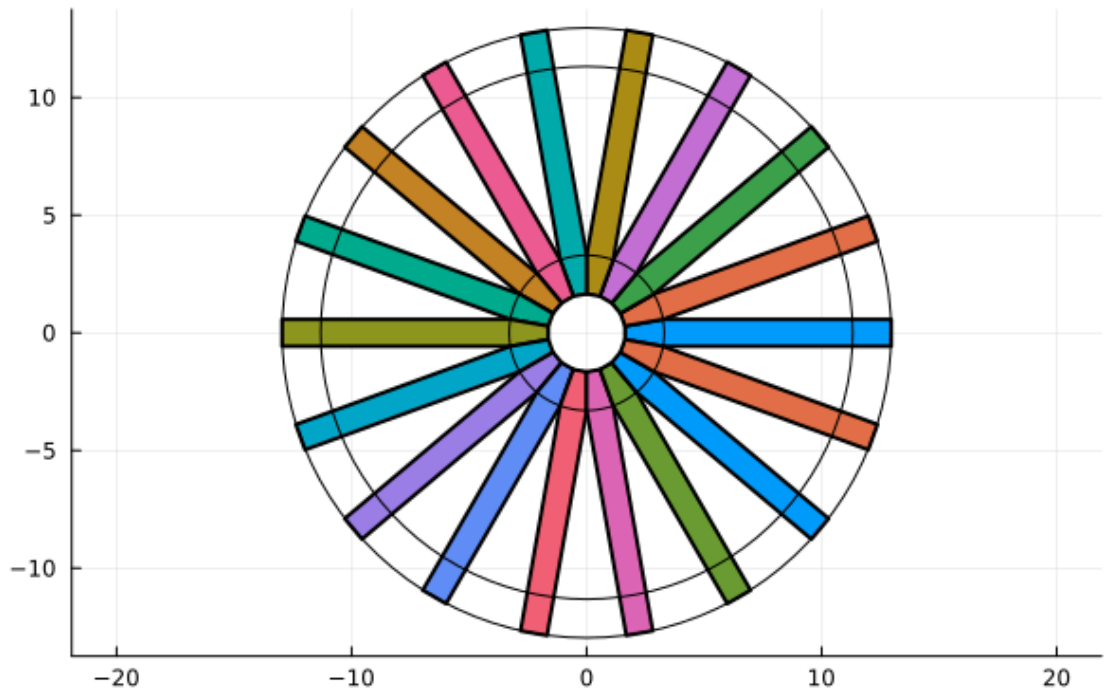


Center Stack stresses

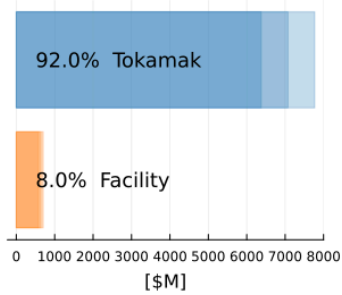




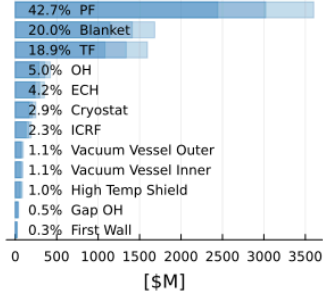
TF coils -- Top view



Direct Capital Cost in 2036 [7.71 \$B]



Tokamak [7.07 \$B]



Facility [0.641 \$B]

